

板子

万用头

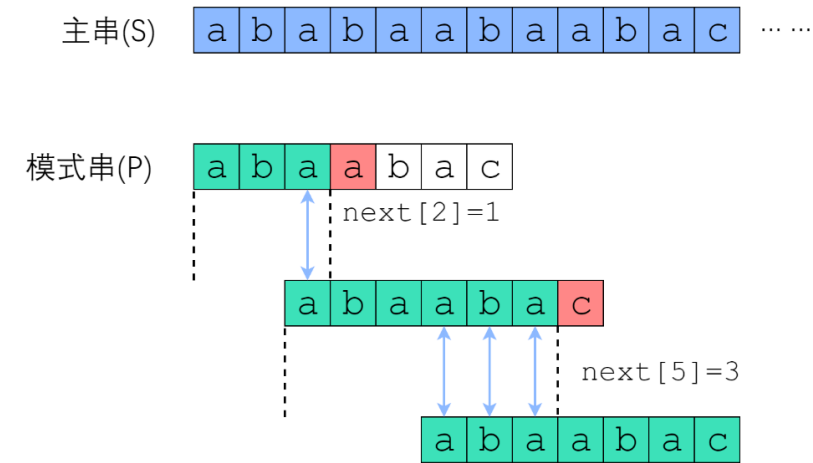
```
1 from collections import deque, defaultdict, Counter
2 from functools import *
3 from bisect import bisect_left, bisect_right
4 from heapq import heappop, heappush
5 from itertools import accumulate, permutations
6 #from more_itertools import distinct_permutations as dispermutions
7 from math import *
8 import sys
9 from typing import List
10 input = sys.stdin.readline
11 def R(): return int(input())
12 def RS():return input().rstrip()
13 def RSS():return [c for c in input().rstrip()]
14 def RLS():return input().rstrip().split()
15 def RR(): return [int(x) for x in input().split()]
16 def get_pre(nums): return list(accumulate(nums, initial=0))
17 from operator import xor,or_,and_
```

KMAP

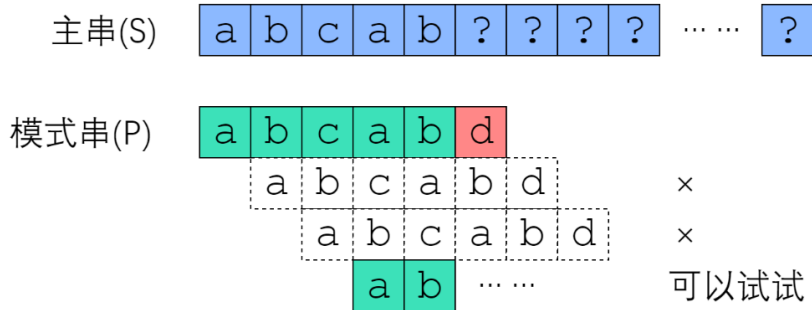
全称 Knuth–Morris–Pratt 算法，是用于在给定的字符串s中查找模式串p是否存在，如果存在，是在那个位置开始。

暴力的做法是，从第一个位置开始遍历s，如果遇到了p的开头，开始——枚举，如果不匹配则s会回退到自己的第 2 个字符，p则回退到自己的开头，然后开始重新比较。

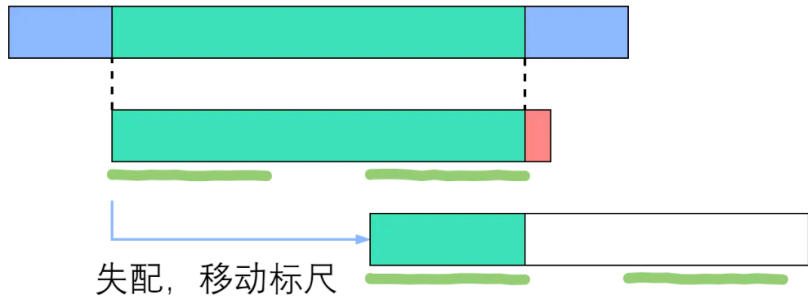
而kmap是减少了查找的次数，即遇到了不匹配的字符，s的位置不会回退，而p会回退。



这里可以看出s不会回退，而p会回退

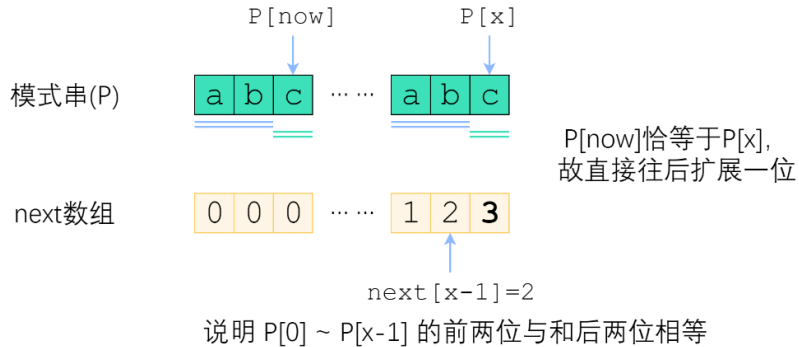


即找到相同的前缀和后缀，把p原来的起始位置换到对应的后缀位置上，减少了无用的枚举，只有从相等的后缀处开始才有可能找到。

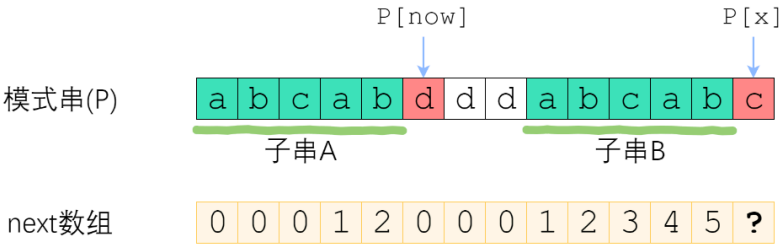


kmap算法的关键在于，构造next数组，next[i]表示P[0]~P[x] 这一段字符串，使得k-前缀恰等于k-后缀的最大的k。

记录now为next[x-1]的结果，如果下一位的字符相同



不同，可以看出子串A与B是一样的，找到子串A的next长度，递归的判断现在的now和x不相同。



arr[i]表示s[:i+1]最长的前缀和后缀相等的长度

```
1 def build(p):
2     # 初始时，自己与自己是等于0的
3     arr=[0]
4     x=1
5     now=arr[x-1]
6     while x<len(p):
7         # 相等，都往后扩展，记录结果
8         if p[now]==p[x]:
9             now+=1
10            x+=1
11            arr.append(now)
12        # 不同的话，递归
13        elif now:
14            now=arr[now-1]
15        # 确实不存在相同的前后缀
16        else:
17            arr.append(0)
18            x+=1
19    return arr
```

```
1 def search(s,p):
2     # 构建前后缀数组
3     arr=build(p)
4     # 标识两个字符串的指针
5     tar,pos=0,0
6     res=[]
7     while tar<len(s):
8         # 相同，往后
9         if s[tar]==p[pos]:
10            tar+=1
11            pos+=1
12        # s与p存在不匹配的
13        elif pos:
14            pos=arr[pos-1]
15        else:
16            # 还没匹配到第一个字符
17            tar+=1
18        # 匹配了一个p，记录结果，这里p要回退而不是重新开始
19        if pos==len(p):
```

```
20 res.append(tar-pos)
21 pos=arr[pos-1]
22 return res
```

字符串哈希

模板

给定一个长度为 n 的字符串，再给定 m 个询问，每个询问包含四个整数 l_1, r_1, l_2, r_2 ，请你判断 $[l_1, r_1]$ 和 $[l_2, r_2]$ 这两个区间所包含的字符串子串是否完全相同。

字符串中只包含大小写英文字母和数字。

输入格式

第一行包含整数 n 和 m ，表示字符串长度和询问次数。

第二行包含一个长度为 n 的字符串，字符串中只包含大小写英文字母和数字。

接下来 m 行，每行包含四个整数 l_1, r_1, l_2, r_2 ，表示一次询问所涉及的两个区间。

注意，字符串的位置从 1 开始编号。

输出格式

对于每个询问输出一个结果，如果两个字符串子串完全相同则输出 **Yes**，否则输出 **No**。

每个结果占一行。

数据范围

$1 \leq n, m \leq 10^5$

输入样例：

```
8 3
aabbbaabb
1 3 5 7
1 3 6 8
1 2 1 2
```

输出样例：

```
Yes
No
Yes
```

逐一比较字符串的每个值超时，对字符串构建前缀和。把字符串看作一个 p 进制的数，比如说 aa 代表一个 $a \times p^1 + a \times p^0$

就这样构造前缀字符串数组，查询一个区间的值的时候，可以通过相减得到字串的数值，但是注意不是直接相减，比如 $aabbc$ ，为了得到 bcc 需要减去前面的 aa 但是 aa 是高进制的需要先左移三位才能减去。

```
1 n,m =[int(x) for x in input().split()]
2 s=input()
3 pre=[0]*(n+1)
4 # 进制，p取131可以防止哈希冲突（技巧点）
```

```
5 p=131
6 # P[i]表示p^i
7 P=[1]
8 # 数值太大会导致计算非常慢，因此要取模
9 mod = 998244353
10 # 构造前缀和，注意取模
11 for i ,c in enumerate(s,1):
12     pre[i]=(pre[i-1]*p+ord(c))%mod
13     P.append((P[-1]*p)%mod)
14 while m:
15     # 注意这里的l,r是从1开始的
16     l1,r1,l2,r2=[int(x) for x in input().split()]
17     # 比较的结果也要取模，防止太大，注意要把后面的数乘上P[字符串的长度]
18     if (pre[r1]-pre[l1-1]*P[r1-l1+1])%mod==(pre[r2]-pre[l2-1]*P[r2-
19 l2+1])%mod:
20         print('Yes')
21     else:
22         print('No')
23 m-=1
```

##Z函数

z[i]表示s[i:](每个后缀)与s的**公共最长前缀**是多少

初始时从1开始比较，找到最长前缀就是暴力匹配，匹配之后会有一段z-box:

Evaluating k=4.
Current index (k=4) is past end of last Z-box (r=1), so Z(4) must be computed explicitly.

Index:	0	1	2	3	4	5	6	7	8	9	10	11
Text:	a	b	a	b	a	a	b	x	a	a	a	z
Text:	a	a	b	c	a	a	a	x	a	a	a	z
Z values:		1	0	0	3							

Z(4) = 3, l=4, r=6, 8 comparisons so far.

Evaluating k=5.
Current index (k=5) is in a previously discovered substring (r=6).

The previously discovered substring starts at k-l (5-4), with Z(1)=1 which is < the remaining substring S[5..6], so Z(5)=Z(1)=1.

No additional comparisons needed.

Index:	0	1	2	3	4	5	6	7	8	9	10	11
Text:	a	b	c	a	a	b	x	a	a	a	a	z
Text:	a	a	b	c	a	a	b	x	a	a	a	z
Z values:		1	0	0	3	1						

Z(5) = 1, l=4, r=6, 8 comparisons so far.

再找下一个s[i:]的过程中如果i落在了z-box中，首先判断z[i-l]是否小于z-box的边界到i的距离，如果小于就确定了z[i]=z[i-l]，反之说明在z-box中的这一段都是匹配的继续往后找，最后更新z-box

```
1 def z(s):
2     n=len(s)
3     z=[0]*n
4     l,r=0,0
5     for i in range(1,n):
6         # 在z-box中并且小于
7         if i<=r and z[i-l]<r-i+1:
8             z[i]=z[i-l]
9         else:
10             # 先初始化为到边界的距离
11             z[i]=max(0,r-i+1)
```

```
12 # 往后找
13 while i+z[i]<n and s[z[i]]==s[i+z[i]]:
14     z[i]+=1
15 # 更新区间
16 if i+z[i]-1>r:
17     l=i
18     r=i+z[i]-1
19 return z
```

如果存在s和p 想找s的每个前缀能匹配多长的p(子串)，由于z函数是自身与自身匹配，这里可以拼接p+s求解z函数 s[i:]能匹配的最长的p即为z[i+len(p)]

对于后缀要倒置s2=p[::-1]+s[::-1]， z2=z(s2) 求解时b=z2[len(p)+len(s)-r(子串的右端点)]

拓扑排序

拓扑排序可以用环和其他的节点区分，拓扑序列不唯一，序列中边起点一定排在终点之前。

拓扑排序的另一个用处是删除无用的叶子节点

给定一个 n 个点 m 条边的有向图，点的编号是 1 到 n ，图中可能存在重边和自环。

请输出任意一个该有向图的拓扑序列，如果拓扑序列不存在，则输出 -1 。

若一个由图中所有点构成的序列 A 满足：对于图中的每条边 (x,y) ， x 在 A 中都出现在 y 之前，则称 A 是该图的一个拓扑序列。

输入格式

第一行包含两个整数 n 和 m 。

接下来 m 行，每行包含两个整数 x 和 y ，表示存在一条从点 x 到点 y 的有向边 (x,y) 。

输出格式

共一行，如果存在拓扑序列，则输出任意一个合法的拓扑序列即可。

否则输出 -1 。

数据范围

$$1 \leq n, m \leq 10^5$$

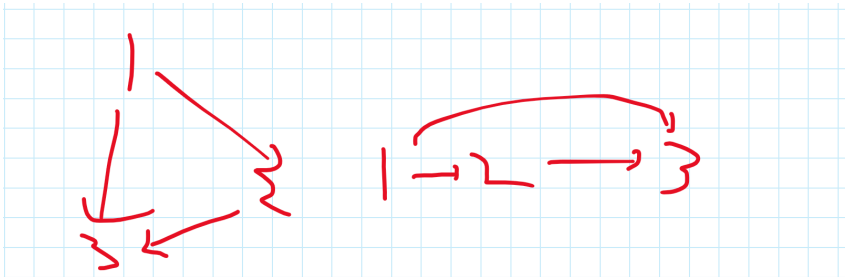
输入样例：

```
3 3
1 2
2 3
1 3
```

输出样例：

```
1 2 3
```

并非只有有向图可以做拓扑排序，对于无向图也可以做，需要加入**入度为1的**。



建图的过程中要记录每个节点的入度，使用节点的过程中首先把入度为0的点加入到队列中，之后遍历队列中的点，对于每一个在队列中的点把与它相连的点之间的边去掉，具体表示为减少下一个点的入度，如果这个点的入度为0了，那么就加入到队列中。最后判断队列的长度是否为N。

```
1 from collections import deque
2 n,m=[int(x) for x in input().split()]
3 path=[set() for _ in range(n)]
4 d=[0]*n
5 q=deque()
6 for _ in range(m):
7     u,v=[int(x)-1 for x in input().split()]
8     # 防止重边
9     if v not in path[u]:
10         path[u].add(v)
11         d[v]+=1
12 # 排序
13 def topsort():
14     # 加入入度为0的点
15     for i,val in enumerate(d):
16         if not val:
17             q.append(i)
18     index=0
19     # 遍历队列中的点
20     while index<len(q):
21         p=q[index]
22         # 去掉边，如果入度变为0则加入
23         for p_ in path[p]:
24             d[p_]-=1
25             if not d[p_]:
26                 q.append(p_)
27         index+=1
28     # 判断是否有拓扑序
29     return len(q)==n
30 if (topsort()):
31     for i in q:
32         print(i+1,end=' ')
33 else:
34     print(-1)
```

基环树

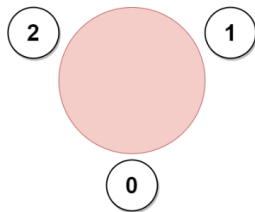
参加会议的最多员工数

一个公司准备组织一场会议，邀请名单上有 n 位员工。公司准备了一张 **圆形** 的桌子，可以坐下 **任意数目** 的员工。

员工编号为 0 到 $n-1$ 。每位员工都有一位 **喜欢** 的员工，每位员工 **当且仅当** 他被安排在喜欢员工的旁边，他才会参加会议。每位员工喜欢的员工 **不会** 是他自己。

给你一个下标从 0 开始的整数数组 `favorite`，其中 `favorite[i]` 表示第 i 位员工喜欢的员工。请你返回参加会议的 **最多员工数目**。

示例 1:



输入: favorite = [2,2,1,2]

输出: 3

解释:

上图展示了公司邀请员工 0, 1 和 2 参加会议以及他们在圆桌上的座位。

没办法邀请所有员工参与会议，因为员工 2 没办法同时坐在 0, 1 和 3 员工的旁边。

注意，公司也可以邀请员工 1, 2 和 3 参加会议。

所以最多参加会议的员工数目为 3。

示例 2:

输入: favorite = [1,2,0]

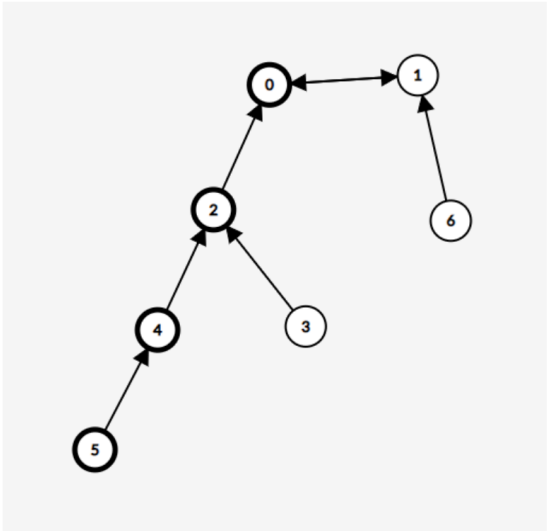
输出: 3

内向基环图，具有 n 个节点 n 条边的连通块，内向指的是连通块中的每个节点的出度为1，外向指的是入度为1，基环图中在基环外可以连接有多个树枝

每个员工只能坐在喜欢的人边上那么座次必须要满足是一个内向基环图

如果基环的大小大于等于3，那么圆桌上能做的人数就是基环的大小，一旦往这个环里向 $x \rightarrow y$ 插入其他的节点，一定会使得 x 不满足条件。

如果基环的大小为2，那么两个点的**两边**可以继续坐人也就是求出两个点的最长链，为此需要构造一个反图，此外在这个座次中可以继续坐入大小为2的基环。



通过一次拓扑排序可以将树枝的入度变为0以此与基环分开，因为图拓扑排序中只能操作树枝，通过构建反图。

```
1 class Solution:
2     def maximumInvitations(self, favorite: List[int]) -> int:
3         n=len(favorite)
4         # 计算入度
5         deg=[0]*n
6         for f in favorite:
7             deg[f]+=1
8         q=deque([i for i,v in enumerate(deg) if not v])
9         r=defaultdict(list)
10        # 将树枝与基环分开，基环是不会入队的
11        while q:
12            x=q.popleft()
13            y=favorite[x]
14            # 构建反图
15            r[y].append(x)
16            deg[y]-=1
17            if not deg[y]:
18                q.append(y)
19        # 求出最长的链
20        def dfs(x):
21            if not r[x]:return 0
22            return max(dfs(y)+1 for y in r[x])
23        a=b=0
24        # 遍历每一个点，找出基环
25        for i,v in enumerate(deg):
26            if not v:continue
27            # 设置为0防止重复访问到
28            deg[i]=0
29            j=favorite[i]
30            res=1
31            # 一直顺着基环找
```

```
32         while j!=i:
33             deg[j]=0
34             j=favorite[j]
35             res+=1
36         # 分类讨论
37         if res>2:
38             a=max(a,res)
39         elif res==2:
40             # 基环长度为2的可以一直累加
41             b+=dfs(i)+dfs(favorite[i])+res
42         return max(a,b)
```

Dijkstra算法

对于有多个状态的问题，需要扩展dls数组，类似DP的做法。
通过将所有的点入队做Dijkstra可以得到每个点到其他点的最短距离

模板

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，所有边权均为正值。
请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 -1 。

输入格式

第一行包含整数 n 和 m 。

接下来 m 行每行包含三个整数 x,y,z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

如果路径不存在，则输出 -1 。

数据范围

$1 \leq n \leq 500$,
 $1 \leq m \leq 10^5$,
图中涉及边长均不超过10000。

输入样例：

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例：

```
3
```

```
1 from math import inf
2
```

```
3 n, m = [int(x) for x in input().split()]
4 path = [[inf] * n for _ in range(n)]
5 dis = [inf] * n
6 dis[0] = 0
7 # 不用标记初始点
8 vis = [False] * n
9 for _ in range(m):
10     u, v, w = [int(x) for x in input().split()]
11     #注意要取最小值, 可能有重边
12     path[u-1][v-1] = min(w, path[u-1][v-1])
13 while True:
14     x = -1
15     # 遍历一遍所有点, 选出其中没有被访问过且力起点最近的点
16     for i in range(n):
17         if not vis[i] and (x < 0 or dis[i] < dis[x]):
18             x = i
19     # 图不连通
20     if x < 0 or dis[x] == inf:
21         print(-1)
22         break
23     # 找到了, 由于求的是单源最短路直接返回
24     if x == n - 1:
25         print(dis[x])
26         break
27     # 标记
28     vis[x] = True
29     # 更新状态
30     for u, w in enumerate(path[x]):
31         dis[u] = min(dis[u], dis[x] + w)
32
```

很明显的看出，每次要遍历所有的点n然后进行这个点边的个数次操作，一次操作中是O(m+n)，最多遍历vis数组的大小，也就是O(n^2)

给定一个 n 个点 m 条边的有向图，图中可能存在重边和自环，所有边权均为非负值。

请你求出 1 号点到 n 号点的最短距离，如果无法从 1 号点走到 n 号点，则输出 -1 。

输入格式

第一行包含整数 n 和 m 。

接下来 m 行每行包含三个整数 x, y, z ，表示存在一条从点 x 到点 y 的有向边，边长为 z 。

输出格式

输出一个整数，表示 1 号点到 n 号点的最短距离。

如果路径不存在，则输出 -1 。

数据范围

$1 \leq n, m \leq 1.5 \times 10^5$ ，

图中涉及边长均不小于 0，且不超过 10000。

数据保证：如果最短路存在，则最短路的长度不超过 10^9 。

输入样例：

```
3 3
1 2 2
2 3 1
1 3 4
```

输出样例：

```
3
```

点数太多而边数有限，使用优先队列优化的算法，优先队列优化指的是使用根堆每次找当前没有被访问过的且距离最小的点。

```
1 from collections import defaultdict
2 from math import inf
3 import heapq
4
5 n, m = [int(x) for x in input().split()]
6 # 这里由于点数太多, 且图是稀疏图因此使用邻接表
7 path = defaultdict(dict)
8 dis = [inf for i in range(n)]
9 dis[0] = 0
10
11 # 建图
12 for _ in range(m):
13     u, v, w = [int(x)-1 for x in input().split()]
14     # 不使用默认字典, 即使是双向边也能用
15     if v in path[u]:
16         path[u][v] = min(path[u][v], w+1)
17     else:
18         path[u][v] = w+1
19
20 # 第一个值作为权重
21 h = [(0, 0)]
22 while h:
23     d, x = heapq.heappop(h)
24     # 当前点之前出过堆
25     if d > dis[x]: continue
26
27     for k, v in path[x].items():
```

```
28         # 严格小于
29         if d+v<dis[k]:
30             dis[k]=d+v
31             heappush(h,(dis[k],k))
32     print(*dis[1:])
```

时间复杂度为O(mn)，根堆中找值的速度为O(logn)，大概一共进行m（边的个数次）

FLoyd算法（多源最短路）

```
1  from math import inf
2  class Solution:
3      def findTheCity(self, n: int, edges: List[List[int]], distanceThreshold:
4          int) -> int:
5          # 初始化为无穷，表示不相连
6          dis=[[inf for _ in range(n)] for _ in range(n)]
7          # 建立直接相连的图
8          for u,v,w in edges:
9              dis[u][v]=dis[v][u]=w
10
11         # dfs(k,i,j)返回从i到j的最短路径，并且这条最短路的中间节点编号都 ≤k。注意中间
12         # 节点不包含 i 和 j。对于选取每个中间节点改为设置一个最大的中间节点，在递归中做到选取每一个中
13         # 间节点
14
15         @cache
16         def dfs(k,i,j):
17             # 当二者之间的中间节点为-1时，表示i 和 j 之间没有任何中间节点，此时最短路长
18             # 度只能是连接 i和 j 的边的边权
19             if k<0:
20                 return dis[i][j]
21             else:
22                 # 对于给定的中间节点，有选或者不选两种可能。
23                 return min(dfs(k-1,i,j),dfs(k-1,i,k)+dfs(k-1,k,j))
24
25         val=inf
26         ans=0
27         # 使用双层循环，外层循环找所有的城市，内层循环找所有的其他的城市，计算二者之间的距
28         # 离如果小于将次数加一
29         for i in range(n):
30             cnt=0
31             for j in range(n):
32                 # 中间节点设置为节点个数，以便选择所有的中间节点
33                 if j!=i and dfs(n-1,i,j)<=distanceThreshold:
34                     cnt+=1
35
36                 # 记录，相等时取最大的 i
37                 if cnt<=val:
38                     val=cnt
39                     ans=i
40
41         return ans
42
43 # 自底向上的写法
44 from math import inf
45 class Solution:
```

```
39     def findTheCity(self, n: int, edges: List[List[int]], distanceThreshold:
40         int) -> int:
41         dis=[[inf for _ in range(n)] for _ in range(n)]
42         for u,v,w in edges:
43             dis[u][v]=dis[v][u]=w
44         # 注意不能忘记标记自己到自己为0
45         for i in range(n):
46             dis[i][i]=0
47         f=dis
48         flag=inf
49         ans=0
50         # 三重循环找最短距离，注意首先是枚举中间节点k
51         for k in range(n):
52             for i in range(n):
53                 for j in range(n):
54                     f[i][j]=min(f[i][j],f[i][k]+f[k][j])
55
56         for i,v in enumerate(f):
57             cnt=0
58             for j,jv in enumerate(v):
59                 # 注意自己到自己不能算
60                 if i!=j and jv<=distanceThreshold:
61                     cnt+=1
62                 # 设置小于等于找最大的编号
63                 if cnt<=flag:
64                     flag=cnt
65                     ans=i
66
67         return ans
```

Kruskal算法

该算法用于**重构**一棵树，从而生成**最小生成树**（Minimum Spanning Tree, MST）是一个在连接所有图的顶点的同时具有最小总权重的树形子图。在一个连通的带权无向图中，生成树是原图的一个子图，它包含了图中的所有顶点，并且是一棵树，没有包含任何环路。树与图的区别是树没有环。

基本思想是**贪心+并查集**，首先对所有的边按**权重**排序然后从最小的边开始遍历，如果二者不连通则连接上同时累加权重，**最后判断是否连通分量等于1（即n个点n-1条边）**

最小生成树算法可以用于求最短路

给定一个 n 个点 m 条边的无向图，图中可能存在重边和自环，边权可能为负数。

求最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

给定一张边带权的无向图 $G=(V,E)$ ，其中 V 表示图中点的集合， E 表示图中边的集合， $n=|V|$ ， $m=|E|$ 。

由 V 中的全部 n 个顶点和 E 中 $n-1$ 条边构成的无向连通子图被称为 G 的一棵生成树，其中边的权值之和最小的生成树被称为无向图 G 的最小生成树。

输入格式

第一行包含两个整数 n 和 m 。

接下来 m 行，每行包含三个整数 u,v,w ，表示点 u 和点 v 之间存在一条权值为 w 的边。

输出格式

共一行，若存在最小生成树，则输出一个整数，表示最小生成树的树边权重之和，如果最小生成树不存在则输出 `impossible`。

数据范围

$1 \leq n \leq 10^5$,
 $1 \leq m \leq 2 * 10^5$,
图中涉及边的边权的绝对值均不超过 1000。

输入样例：

```
4 5
1 2 1
1 3 2
1 4 3
2 3 2
3 4 4
```

输出样例：

```
1 class UF:
2     def __init__(self,n):
3         self.p=[i for i in range(n)]
4         self.count=n
5         self.size=[1 for _ in range(n)]
6     def find(self,x):
7         d=self
8         if d.p[x]!=x:
9             d.p[x]=d.find(d.p[x])
10        return d.p[x]
11    def union(self,x,y):
12        d=self
13        u=d.find(x)
14        v=d.find(y)
15        if u==v:
16            return
17        if d.size[u]<d.size[v]:
18            d.size[v]+=d.size[u]
19            d.p[u]=d.p[v]
20        else:
21            d.size[u]+=d.size[v]
22            d.p[v]=d.p[u]
23        d.count-=1
24    def connect(self,x,y):
25        d=self
```

```
26         u=d.find(x)
27         v=d.find(y)
28         return v==u
29     def judge(self):
30         return self.count==1
31 n,m=[int(x) for x in input().split()]
32 edges=[]
33 for _ in range(m):
34     edges.append(list(map(int,input().split())))
35 edges.sort(lambda x:x[2])
36
37 uf=UF(n)
38 res=0
39 for i in range(m):
40     u,v,w =edges[i]
41     u-=1
42     v-=1
43     if not uf.connect(u,v):
44         uf.union(u,v)
45         res+=w
46 if uf.judge():
47     print(res)
48 else:
49     print('impossible')
```

时间复杂度为O(mlogm+m)

欧拉回路

欧拉路径定义为：如果一条路径可以经过图中所有的边恰好一次，那么这条路径就是欧拉路径。数学家欧拉为了解决著名的哥尼斯堡七桥问题提出的。

欧拉路径又分为欧拉回路——路径的**起点与终点一样**，欧拉开路(通路)——**起点和终点不同**。

对于无向图：

- 欧拉回路：图是连通的，每个顶点的度数均是偶数
- 欧拉开路：图是连通的，并且只有两个顶点的度是奇数

对于有向图：

- 欧拉回路：图是连通的，每个顶点的入度等于出度
- 欧拉开路：图是连通的，只有两个顶点的入度出度不同，其中一个入度比出度大一，其为欧拉路径的终点；另一个入度比出度小一，其为欧拉路径的起点

重新安排行程

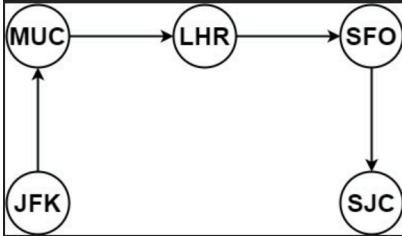
给你一份航线列表 `tickets`，其中 `tickets[i] = [fromi, toi]` 表示飞机出发和降落的机场地点。请你对该行程进行重新规划排序。

所有这些机票都属于一个从 `JFK`（肯尼迪国际机场）出发的先生，所以该行程必须从 `JFK` 开始。如果存在多种有效的行程，请你按字典序排序返回最小的行程组合。

- 例如，行程 `["JFK", "LGA"]` 与 `["JFK", "LGB"]` 相比就更小，排序更靠前。

假定所有机票至少存在一种合理的行程。且所有的机票 必须都用一次 且 只能用一次。

示例 1:



输入: `tickets = [["MUC", "LHR"], ["JFK", "MUC"], ["SFO", "SJC"], ["LHR", "SFO"]]`
输出: `["JFK", "MUC", "LHR", "SFO", "SJC"]`

本题需要找出经过所有边一次的路径，那么就是求解欧拉路径，题目保证有解，那么“JFK”一定是欧拉路径的起点。

求解欧拉路径使用Hierholzer算法：

1. 从起点出发，进行深度优先搜索。
2. 每次沿着某条边从某个顶点移动到另外一个顶点的时候，都需要删除这条边。
3. 如果没有可移动的路径，则将所在节点加入到栈中，并返回。

最后求解出的是倒序的欧拉路径

本题为了得到最小的字典序，需要向将边排序

```
1 class Solution:
2     def findItinerary(self, edges: List[List[str]]) -> List[str]:
3         deg=defaultdict(list)
4         # 使用堆排序
5         for u,v in edges:
6             deg[u].append(v)
7         for k in deg:
8             heapify(deg[k])
9         ans=[]
10
11         def dfs(root):
12             # 不断往后找
13             while deg[root]:
14                 # 拆边
```

```
15         dfs(heapop(deg[root]))
16         # 没有边了，加入自己
17         ans.append(root)
18         dfs("JFK")
19         # 求解出的是倒序
20         return ans[::-1]
```

有向图的强连通分量

一般用于有向图的缩点

定义

连通：无向图中可以从任意点i可以到达任意点j

强连通：有向图中可以从任意点i可以到达任意点j

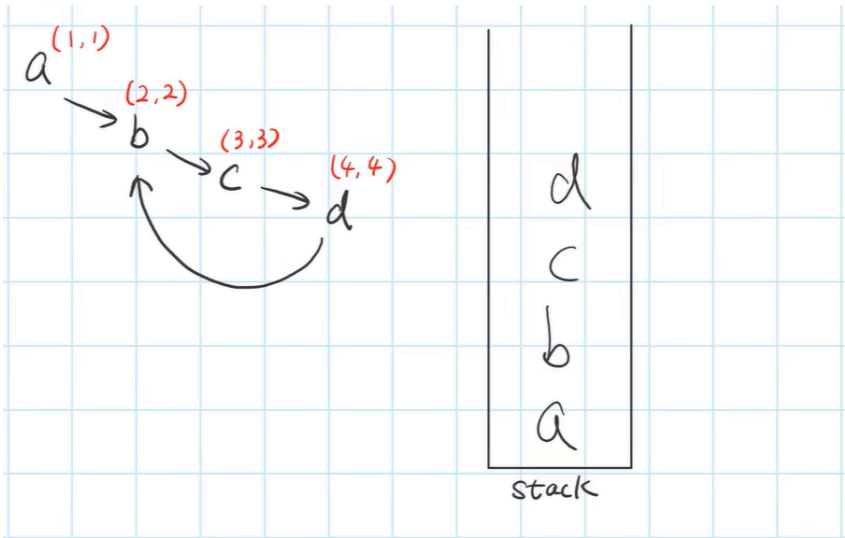
弱连通：有向图看作是无向图时，可以从任意点i可以到达任意点j

dfn[x]定义为遍历过程中到达x的最早时间

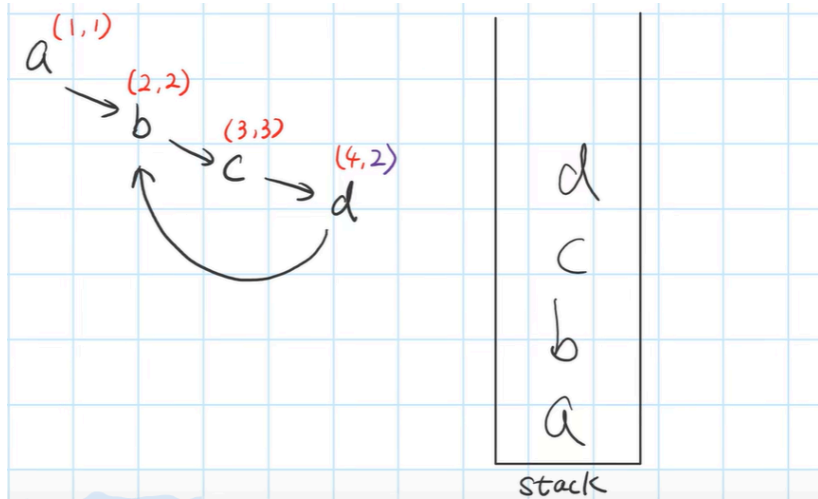
low[x]定义为以x可以回溯到的最早时间点

过程

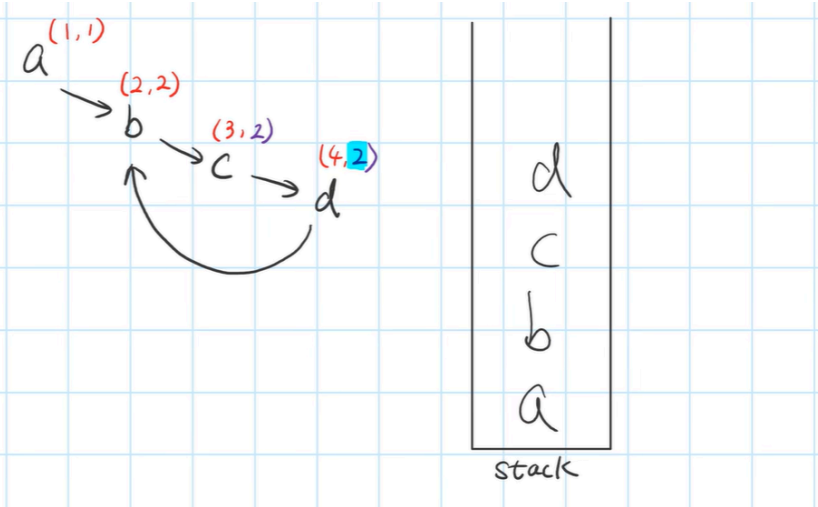
在dfs遍历的过程中将元素存储到栈中



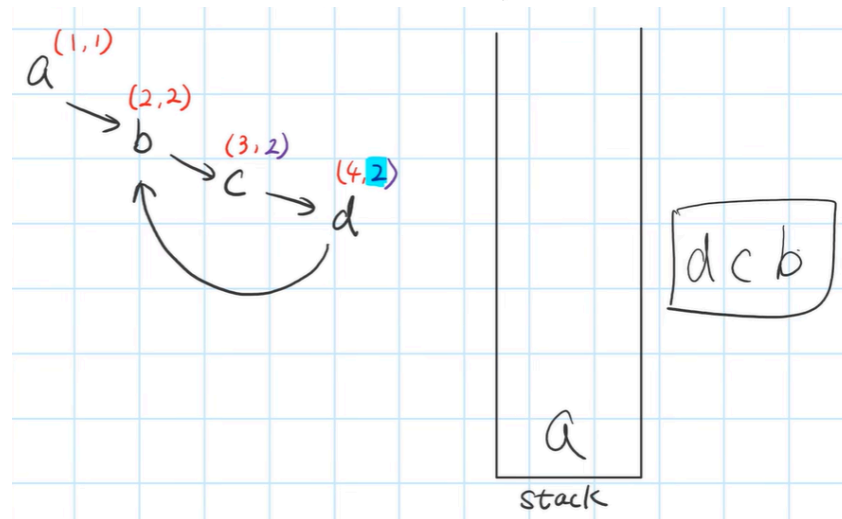
d能回溯到更早的时间点, 更新low[d]=low[b]



d返回到c, c跟更新其Low为最小的



随后c返回到b, b的low值不会在变小了, 并且b的dfn==low值, 此时弹出栈中的值直到弹出b, 因此dcb就组成了一个强连通分量



施法所需最低符文数量

Alice 刚刚从巫师学校毕业, 并且希望施展一个魔法咒语来庆祝。魔法咒语包含某些需要集中魔力的焦点, 其中一些焦点含有作为咒语能量源的魔法水晶。焦点可以通过 **有向符文** 进行连接, 这些符文将魔力流从一个焦点传输到另一个焦点。

给定一个整数 n 表示焦点的数量, 以及一个整数数组 `crystals`, 其中 `crystals[i]` 表示有魔法水晶的焦点。同时给定两个整数数组 `flowFrom` 和 `flowTo`, 表示存在的 **有向符文**。第 i^{th} 个符文允许魔力流从焦点 `flowFrom[i]` 传输到焦点 `flowTo[i]`。

你需要找到 Alice 必须添加到她的咒语中的定向符文数量, 使得每个焦点要么:

- 包含一个魔法水晶。
- 从其它焦点 **接收** 魔力流。

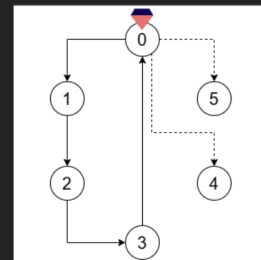
返回她所需要添加的 **最少** 有向符文数量。

示例 1:

输入: $n = 6$, `crystals = [0]`, `flowFrom = [0,1,2,3]`, `flowTo = [1,2,3,0]`

输出: 2

解释:



对于水晶访问不到的分量进行缩点, 缩点后所有入度为0的点就是要添加的边的数量

```
1 class Solution:
2     def minRunesToAdd(self, n: int, crystals: List[int], flowFrom:
List[int], flowTo: List[int]) -> int:
3         g=defaultdict(list)
4         deg=[0]*n
5         # 拓扑排序, 防止从一条链的末尾开始不断重复访问
6         for u,v in zip(flowFrom,flowTo):
7             g[u].append(v)
8             deg[v]+=1
9         d=deque()
10        for i,x in enumerate(deg):
11            if not x:d.append(i)
12
13        curr=0
14        while curr<len(d):
15            x=d[curr]
16            for nx in g[x]:
17                deg[nx]-=1
18                if not deg[nx]:d.append(nx)
19            curr+=1
20        nums=list(d)
21
22        vis=set(crystals)
23        d=deque(crystals)
24        # 连通分量的标号
25        gid=n
26
27        # 有哪些新的连通分量
28        group=defaultdict(set)
29        # 点属于哪个连通分量
30        m={}
31
32        # bfs标记可以水晶可以访问到的点
33        while d:
34            x=d.popleft()
35            for nx in g[x]:
36                if nx not in vis:
37                    vis.add(nx)
38                    d.append(nx)
39        def tarjan(root,t):
40            nonlocal gid
41            dfn[root]=low[root]=t
42            d.append(root)
43            for nx in g[root]:
44                if nx not in dfn:
45                    tarjan(nx,t+1)
46                    low[root]=min(low[root],low[nx])
47            else:
48                low[root]=min(low[root],dfn[nx])
49        # 可以弹出
50        if low[root]==dfn[root]:
51            while d:
52                x=d.pop()
53                group[gid].add(x)
54                m[x]=gid
```

```
55         if x==root:break
56         gid+=1
57
58         dfn={}
59         low={}
60         d=deque()
61         # 对访问不到的分量进行缩点, 通过拓扑序可以先访问链的首部, 防止从尾部开始重复访问
62         for i in nums:
63             # 通过m记录可以防止重复对同一个连通块缩点
64             if i not in vis and i not in m:
65                 tarjan(i,0)
66         # 并非所有的点都会被拓扑排序到
67         for i in range(n):
68             if i not in vis and i not in m:
69                 tarjan(i,0)
70         # 统计缩点后的入度
71         deg=defaultdict(int)
72         for u,v in zip(flowFrom,flowTo):
73             if u in m:u=m[u]
74             if v in m:v=m[v]
75             # 缩点后属于同一点, 跳过
76             if u==v:continue
77             deg[v]+=1
78         ans=0
79         # 找出缩点后入度为0的, 且不会被访问到的点
80         for i in group:
81             if i in vis:continue
82             if not deg[i]:ans+=1
83         return ans
```

##

无向图的双连通分量

讲解视频https://www.bilibili.com/video/BV1Q7411e7bM?vd_source=63004f7ee2f738d6adc086e00ebcede2&spm_id_from=333.788.player.switch&p=4

进行缩点后可以看作是一颗树

定义

桥: 对于连通的图如果删除一条边后图变得不连通了

割点: 如果删掉这个点后整个图会变得不连通, 割点的判断比较反常, 有时是看不出来的, 需要按照dfn和low判断来决定

dfn[x]定义为遍历过程中到达x的最早时间

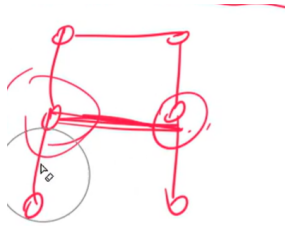
low[x]定义为以x可以回溯到的最早时间点

在无向图中从子节点到父节点的边不予处理, 防止low[y]=dfn[x]导致无法判断桥。

双连通分量分为：边双连通分量e-dcc(极大的不含有桥的连通分量)，点双连通分量v-dcc(极大的不包含割点的连通分量)

对于v-dcc找到割点后，对其他点做bfs遇到割点就加入结果，但不加入队列，这样就能找到所有的连通分量。

两个割点之间的边不一定是桥



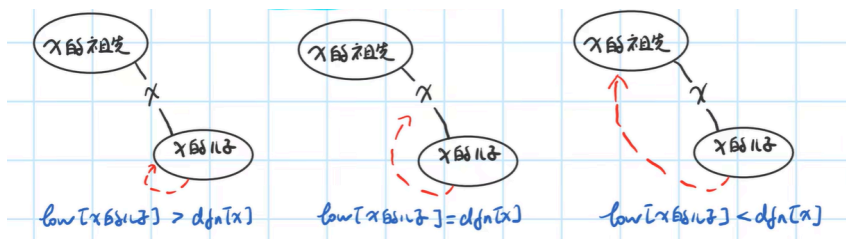
桥的两个端点不一定是割点



如果x是割点：

这里的root指的是整个回溯的起点

1. $x \neq \text{root}$ & x 有子节点 & $\text{low}[x.\text{child}] \geq \text{dfn}[x]$ ，即子节点不能回溯到x的祖先，删除会导致不连通，因此是一个割点。



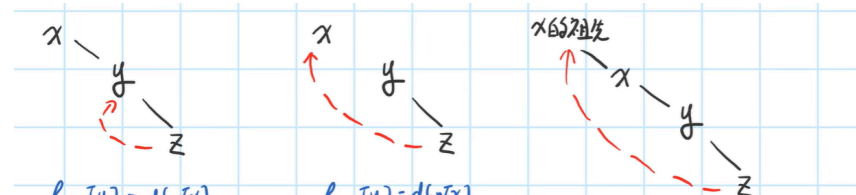
2. 当x是根节点且子节点 ≥ 2

这里的子节点指的是dfs搜索过程中的子节点。左图中有两个子节点，右图中只有一个子节点。



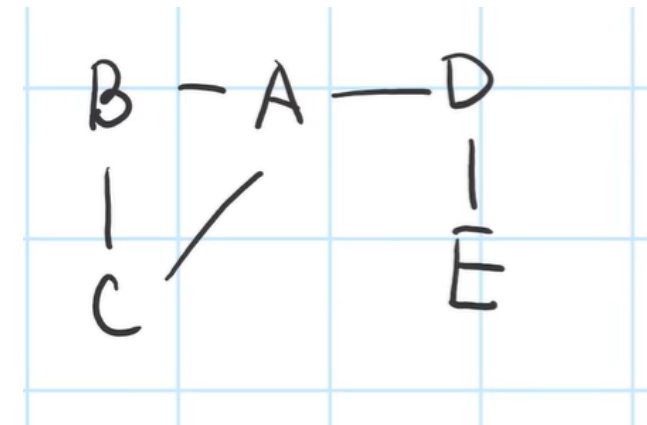
如果边是桥：

$x \rightarrow y$ 是桥有 $\text{low}[y] > \text{dfn}[x]$ ，这样就不会回溯到比x更早的时间点，那么删除 $x \rightarrow y$ 就会导致不连通。

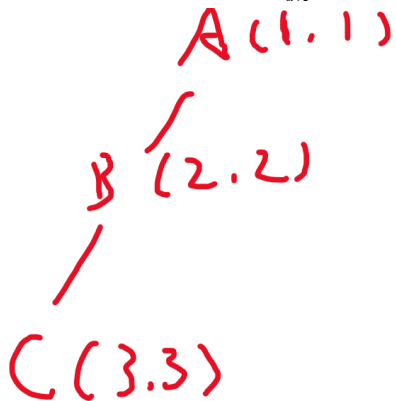


tarjan算法是基于dfs搜索的过程，其中每条边只会访问一次。

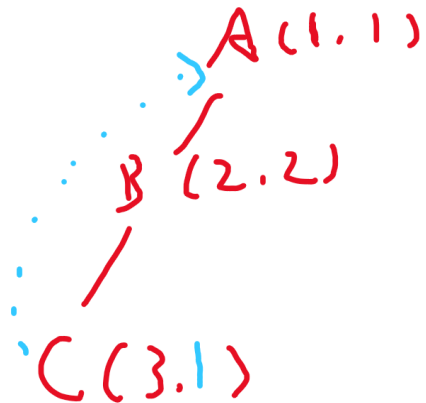
过程



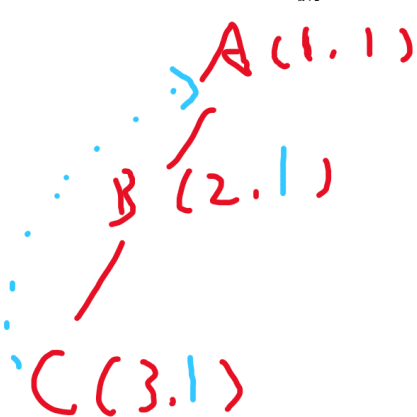
假设从A开始dfs，接下来先找B再找C



无向图中不会往回找父节点，但经过一条边到父节点的父节点是没有问题的，因此可以从C到A，那么 $low[C]=dfn[A]=1$



当前回溯结束，从C返回到B，C的Low能更小，自然B的low也能更小，因为可以从B到C， $low[b]=low[c]=1$ ，这里的B应该是找子节点中最小的low



返回到A后继续找D和E，也是一样的过程

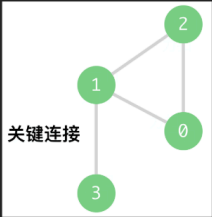
查找集群内的关键连接

力和数据中心有 n 台服务器，分别按从 0 到 $n-1$ 的方式进行了编号。它们之间以 服务器到服务器 的形式相互连接组成了一个内部集群，连接是无向的。用 `connections` 表示集群网络，`connections[i] = [a, b]` 表示服务器 `a` 和 `b` 之间形成连接。任何服务器都可以直接或者间接地通过网络到达任何其他服务器。

关键连接 是在该集群中的重要连接，假如我们将它移除，便会导致某些服务器无法访问其他服务器。

请你以任意顺序返回该集群内的所有 **关键连接**。

示例 1:



输入: $n = 4$, `connections = [[0,1],[1,2],[2,0],[1,3]]`
输出: `[[1,3]]`
解释: `[[3,1]]` 也是正确的。

示例 2:

输入: $n = 2$, `connections = [[0,1]]`
输出: `[[0,1]]`

提示:

- $2 \leq n \leq 10^5$

关键路径——删除掉图就无法连通，那么就是桥，因此使用Tarjan找到桥即可。

```
1 class Solution:
2     def criticalConnections(self, n: int, edges: List[List[int]]) ->
List[List[int]]:
3         dfn=[-1]*n
4         low=[-1]*n
5         ans=[]
6         # 建图
7         g=[[] for _ in range(n)]
8         for u,v in edges:
9             g[u].append(v)
10            g[v].append(u)
11
12        def Tarjan(root,fa,t):
13            # 更新时间戳
14            dfn[root]=t
15            low[root]=t
16            # 找下一点
17            for nx in g[root]:
18                # 不能往回走
19                if nx==fa:continue
20                # 没访问过
21                if dfn[nx]==-1:
22                    Tarjan(nx,root,t+1)
23                    # 更新low为最小的
24                    low[root]=min(low[root],low[nx])
25                    # 判断是否是桥
26                    if low[nx]>dfn[root]:
27                        ans.append([root,nx])
28            else:
29                low[root]=min(dfn[nx],low[root])
30        Tarjan(0,-1,0)
31        return ans
```

二分图

- 二分图就是可以把所有的点分为两部分，这两部分内部不存在边，而边只存在于两个图之间。
- 二分图一定不含奇数环，图中只有偶数环

二分图染色

判断二分图

使用红蓝染色法进行判断，用bfs实现，使用dfs爆栈，注意自边不能忽略

```
1 from collections import deque
2 n, m = [int(x) for x in input().split()]
3 path = defaultdict(set)
4 color = [-1] * n
5
```

```
6 for _ in range(m):
7     u, v = [int(x) - 1 for x in input().split()]
8     # 无向图
9     path[u].add(v)
10    path[v].add(u)
11
12    def bfs(i, c):
13        d = deque()
14        d.append(i)
15        # 标记颜色
16        color[i] = c
17        while d:
18            sz = len(d)
19            for _ in range(sz):
20                curr = d.popleft()
21                for j in path[curr]:
22                    # 如果没有染色，标记颜色为相反的加入队列
23                    if color[j] == -1:
24                        color[j] = 1 - c
25                        d.append(j)
26                    # 如果染了色并且颜色矛盾
27                    elif color[j] == c:
28                        return False
29            # 变更颜色，1变为0，0变为1，通过用1减去快速转换
30            c = 1 - c
31        return True
32
33    # 这里枚举每个点，因为图不一定连通
34    for i in range(n):
35        if color[i] == -1:
36            if not bfs(i, 0):
37                print('No')
38                break
39    else:
40        print('Yes')
```

将节点分成尽可能多的组

给你一个正整数 n ，表示一个 **无向** 图中的节点数目，节点编号从 1 到 n 。

同时给你一个二维整数数组 `edges`，其中 `edges[i] = [ai, bi]` 表示节点 a_i 和 b_i 之间有一条 **双向** 边。注意给定的图可能是不连通的。

请你将图划分为 m 个组（编号从 1 开始），满足以下要求：

- 图中每个节点都只属于一个组。
- 图中每条边连接的两个点 `[ai, bi]`，如果 a_i 属于编号为 x 的组， b_i 属于编号为 y 的组，那么 $|y - x| = 1$ 。

请你返回最多可以将节点分为多少个组（也就是最大的 m ）。如果没办法在给定条件下分组，请你返回 -1 。

示例 1：

Group 1

Group 2

Group 3

Group 4

输入: $n = 6$, `edges = [[1,2],[1,4],[1,5],[2,6],[2,3],[4,6]]`
输出: 4
解释: 如上图所示,
- 节点 5 在第一个组。
- 节点 1 在第二个组。
- 节点 2 和节点 4 在第三个组。
- 节点 3 和节点 6 在第四个组。
所有边都满足题目要求。
如果我们创建第五个组, 将第三个组或者第四个组中任何一个节点放到第五个组, 至少有一条边连接的两个节点所属的组编号不符合题目要求。

- `1 <= n <= 500`
- `1 <= edges.length <= 104`
- `edges[i].length == 2`
- `1 <= ai, bi <= n`
- `ai != bi`
- 两个点之间至多只有一条边。

从特殊开始思考，如果无向图中没有环，那么就是一颗树，通过枚举每个点做bfs得到的最大的深度就是最多的分组。

如果图中存在环，如果是奇环那么一定无解，如果是偶数则可以按照树的方式求解，因此要判断图中是否只有偶数环(通过红蓝染色法)

注意图不一定连通所以对每个连通块操作

```
1 class Solution:
```

```
2 def magnificentSets(self, n: int, edges: List[List[int]]) -> int:
3     g=[[] for _ in range(n)]
4     color=[-1]*n
5     # 建图
6     for u,v in edges:
7         g[u-1].append(v-1)
8         g[v-1].append(u-1)
9     def br(i,c):
10         color[i]=c
11         d=deque([i])
12         while d:
13             for _ in range(len(d)):
14                 curr=d.popleft()
15                 # 这里需要记录每个连通块有哪些点
16                 node.append(curr)
17                 for nx in g[curr]:
18                     if color[nx]==-1:
19                         color[nx]=1-c
20                         d.append(nx)
21                     # 颜色不能矛盾
22                     elif color[nx]==c:
23                         return False
24                 # 颜色转换
25                 c=1-c
26         return True
27
28     def bfs(root):
29         cnt=0
30         vis=set([root])
31         d=deque([root])
32
33         while d:
34             cnt+=1
35             for _ in range(len(d)):
36                 curr=d.popleft()
37                 for nx in g[curr]:
38                     if nx not in vis:
39                         vis.add(nx)
40                         d.append(nx)
41         return cnt
42
43     ans=0
44     for i in range(n):
45         # 找连通块
46         if color[i] == -1:
47             node=[]
48             # 找到连通块且块中没有奇数环
49             if not br(i, 0):return -1
50             # 枚举块中的每个点做bfs
51             ans+=max(bfs(i) for i in node)
52     return ans
```

二分图最大匹配

使用匈牙利算法，时间复杂度为 $O(mn)$

匹配指的是任意两条边都不会依附于同一个点，而最大匹配找的就是最多的匹配数。

基本的思路是把匹配的过程视为找男女朋友，遍历所有的男生找与他相匹配的女生，如果这个女生没有男朋友就是为他们两个匹配，如果有男朋友看看它的男朋友是否能换一个女生，如能皆大欢喜反之不能匹配。

给定一个二分图，其中左半部包含 n_1 个点（编号 $1 \sim n_1$ ），右半部包含 n_2 个点（编号 $1 \sim n_2$ ），二分图共包含 m 条边。

数据保证任意一条边的两个端点都不可能在同一部分中。

请你求出二分图的最大匹配数。

二分图的匹配：给定一个二分图 G ，在 G 的一个子图 M 中， M 的边集 $\{E\}$ 中的任意两条边都不依附于同一个顶点，则称 M 是一个匹配。

二分图的最大匹配：所有匹配中包含边数最多的一组匹配被称为二分图的最大匹配，其边数即为最大匹配数。

输入格式

第一行包含三个整数 n_1 、 n_2 和 m 。

接下来 m 行，每行包含两个整数 u 和 v ，表示左半部点集中的点 u 和右半部点集中的点 v 之间存在一条边。

输出格式

输出一个整数，表示二分图的最大匹配数。

数据范围

$1 \leq n_1, n_2 \leq 500$,
 $1 \leq u \leq n_1$,
 $1 \leq v \leq n_2$,
 $1 \leq m \leq 10^5$

```
1 from collections import defaultdict
2
3 n1, n2, m = [int(x) for x in input().split()]
4 path = defaultdict(set)
5 # 建图只需要建立一边，另一部分不会被考虑
6 for _ in range(m):
7     u, v = [int(x) - 1 for x in input().split()]
8     path[u].add(v)
9
10 # 构建数组，表示当前女生的男朋友是
11 match = [-1] * n2
12
13 # 该函数返回当前男生是否能匹配成功
14 def find(u):
```

```
15 # 找中意的女生
16 for k in path[u]:
17     # 这个女生没有被考虑过
18     if not st[k]:
19         # 标记
20         st[k] = True
21         # 如果这个女生没有男朋友或者说可以叫他的男朋友换一个（递归函数实现），成功
22         if match[k]==-1 or find(match[k]):
23             # 记录
24             match[k]=u
25             return True
26     return False
27
28 res = 0
29 # 遍历所有的男生
30 for i in range(n1):
31     # 该数组标记女生有没有考虑过，每次循环更新一遍
32     st = [False] * n2
33     if find(i):
34         # 累加
35         res += 1
36 print(res)
```

置换环

逐层排序二叉树所需的最少操作数目

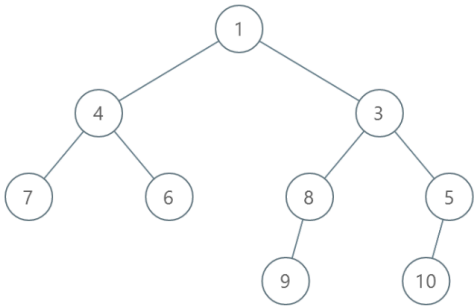
给你一个 **值互不相同** 的二叉树的根节点 `root` 。

在一步操作中，你可以选择 **同一层** 上任意两个节点，交换这两个节点的值。

返回每一层按 **严格递增顺序** 排序所需的最少操作数目。

节点的 **层数** 是该节点和根节点之间的路径的边数。

示例 1：



输入： `root = [1,4,3,7,6,8,5,null,null,null,null,9,null,10]`
输出： 3
解释：

- 交换 4 和 3 。第 2 层变为 [3,4] 。
- 交换 7 和 5 。第 3 层变为 [5,6,8,7] 。
- 交换 8 和 7 。第 3 层变为 [5,6,7,8] 。

共计用了 3 步操作，所以返回 3 。

可以证明 3 是需要的最少操作数目。

问题的关键在于一串数字如何确定通过交换两个数字达到有序的最少次数是多少， 这个问题要用到置换环（技巧点）：比如一串数字**1 0 3 4 2**，其中后三位和前两位的交换是互不干涉的，可以被分为两个环，而找到环的技巧是以当前值为下标，不断在数组中查找直到，回到原位，这就是一个环，最少的交换次数是环中的数字个数减一（或者是数组中元素的个数减去环的个数），而本题中数组并不使从0到n的而是随机开始的，因此需要离散化处理（技巧点），把5 6 7 8 变为0 1 2 3， 最简单的做法是将值排序后用二分搜索。

本题中的bfs采用滚动数组的形式，没有选择双向队列。

```
1 class Solution:
2     def minimumOperations(self, root: Optional[TreeNode]) -> int:
3         ans=0
4         q=[root]
5         while q:
6             # 滚动数组形式，用tmp操作，q变为储存下一层内容的数组
7             tmp=q
8             # a中存储的是节点的值
9             a=[]
10            q=[]
```

```
11         for node in tmp:
12             a.append(node.val)
13             if node.left:
14                 q.append(node.left)
15             if node.right:
16                 q.append(node.right)
17         # 离散化，排序二分
18         b=sorted(a)
19         a=[bisect_left(b,v) for v in a]
20         n=len(a)
21         # 最小的操作次数可以化简为，数字个数之和减去环的个数
22         ans+=n
23         # 使用vis数组标记是否访问过
24         vis=[False]*n
25         for p in a:
26             if vis[p]:continue
27             # 一直往下找
28             while not vis[p]:
29                 vis[p]=True
30                 p=a[p]
31             ans-=1
32         return ans
```

数学知识

偶数和奇数

从1到n中**奇数**的个数为 $(n+1)/2$ ，**偶数**的个数为 $n-(n+1)/2$ ，通过**异或**两个数可以判断两个数的奇偶性是否相同($==1?$)

取模

想迭代计算 $x=10x+1$ ，为防止结果太大需要取模，在迭代的过程中 $10*x+1$ 中的x可以直接取上一次迭代后取模的结果

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$

$$(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$$

质数（素数）

对于质数相关的题目经常打表计算，构建质数表的效率至关重要，如果选择遇到一个数字就去从2开始遍历它，然后在遍历中判断是不是质数，这样的效率非常低。

注意使用质数表的时候，要把表的运算放到外面进行，因为一道题有很多个测试数据，放在里面每次测试都需要算，放在外面只需要算一次。

注意1e7无法打表

找因数质数什么的都可以从开方处找（可以取到）。

梅森素数指的是x是质数，x-1也是质数。

两个互质的数，不能组合出的最大的数是AB-A-B

分解质因数

一个数的质因数指的是它的因数都是由质数组成的，比如说6就是有2和3组成的，而8是由三个2组成的。

找质因数的过程类似于试除法，从小到大找每个数如果他是一个因数，通过把给定的数一直整除这个因数直到不再是因数来排除掉这个因数的倍数从而保证所有的因数都是质数，然后继续找下一个数。注意质因数中只会有一个大于根号n，因此可以遍历到根号n，如果最后剩下的数大于1那么这就是余下的质数。

给定 n 个正整数 a_i ，将每个数分解质因数，并按照质因数从小到大的顺序输出每个质因数的底数和指数。

输入格式

第一行包含整数 n 。

接下来 n 行，每行包含一个正整数 a_i 。

输出格式

对于每个正整数 a_i ，按照从小到大的顺序输出其分解质因数后，每个质因数的底数和指数，每个底数和指数占一行。

每个正整数的质因数全部输出完毕后，输出一个空行。

数据范围

$1 \leq n \leq 100$,
 $2 \leq a_i \leq 2 \times 10^9$

输入样例：

```
2
6
8
```

输出样例：

```
2 1
3 1

2 3
```

```
1 from math import sqrt
2 n=int(input())
3 def cal(x):
4     # 找每个数判断是不是因数
5     for i in range(2,int(sqrt(x)+1)):
6         if x%i==0:
7             s=0
```

```
8         # 如果是，把他的倍数全部消掉
9         while x%i==0:
10             x/=i
11             s+=1
12         ans.append([i,s])
13     # 如果最后留下一个那就是剩余的质因数
14     if x>1:
15         ans.append([x,1])
16
17 for _ in range(n):
18     x=int(input())
19     ans=[]
20     cal(x)
21     for i in ans:
22         print(*i)
23     print()
```

埃氏筛

对于给定的数据范围，从2开始找质数，如果这个数是质数，那么它的冥次一定不是质数，在判断表中标记

```
1 MAX=10**6+1
2 # 记录这个数是不是质数，初始为True
3 is_prime=[True]*MAX
4 # 注意0和1的特判
5 is_prime[0]=False
6 is_prime[1]=False
7 # 记录质数
8 prime=[]
9 # 对于给定的数据范围，从2开始找质数
10 for i in range(2,MAX):
11     if is_prime[i]:
12         # 如果是质数加入，并把他的冥次标记
13         prime.append(i)
14         # 这里的优化是设置步长并从i**2开始
15         for j in range(i*i,MAX,i):
16             is_prime[j]=False
```

在1e6的范围内速度更快

下述式子的时间复杂度为O(nlogn)

```
1 for i in range(1,MX):
2     for j in range(i,MX,i):
```

线性筛（欧拉筛）

每个数只会被他的最小质因数划掉，每个数乘上所有小于等于它最小质因数的质数来筛别的数。

```
1 MAX=(10**6)+1
2 is_prime=[True]*MAX
3 is_prime[0]=is_prime[1]=False
4 prime=[]
5 for i in range(2,MAX):
6     if is_prime[i]:
7         prime.append(i)
8         # 要对每个数进行筛选，不是只对质数筛选，下面的循环要放到外面
9     for p in prime:
10        # 防止越界
11        if p*i>=MAX:
12            break
13        is_prime[p*i]=False
14        if i%p==0:
15            break
```

试除法

判断是否是质数使用试除法，对于不是质数的数，它的因数都是成对存在的。因此判断质数的时候只需要对一边判断即可。

```
1 from math import sqrt
2 n=int(input())
3 ans=[]
4 def cal(x):
5     # 特判1
6     if x==1:
7         return False
8     for i in range(2,int(sqrt(x)+1)):
9         if x%i==0:
10            return False
11    return True
12 for _ in range(n):
13     x=int(input())
14     if cal(x):
15         ans.append('Yes')
16     else:
17         ans.append('No')
18 for i in ans:
19     print(i)
```

更好的方法是用打表判断出的质数，而不是一个一个看，因为本来也就只有质数能起作用。

对于10^9 (30000)的数据可以开5*10^4

```
1 p=[]
```

```
2 MX=(10**4)*5
3 is_prime=[True]*MX
4 for i in range(2,MX):
5     if is_prime[i]:
6         p.append(i)
7         for j in range(i**2,MX,i):is_prime[j]=False
8 n=len(p)
9
10 i=0
11 cnt=0
12 x=int(input())
13 while x>1 and j<n:
14     v=p[i]
15     if x%v==0:
16         while x%v==0:
17             cnt+=1
18             x//=v
19     j+=1
20 if x>1:
21     cnt+=1
```

质数个数

判断1~n中质因数p的个数

[1,n]中p的倍数有 $n1=n//p$ 个，这些数贡献了n1个质因子p， p^2 的倍数有 $n//p^2$ 个，由于这些数已经是p的倍数了，为了不重复统计p的个数，这里仅考虑额外贡献的质因子个数，即这些数额外贡献了n2个质因子p，以此类推有

[1,n] 中质因子 p 的个数为

$$\sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor$$

因数

一个数的因数个数大概是立方级别

因数个数

给定 n 个正整数 a_i ，请你输出这些数的乘积的约数个数，答案对 $10^9 + 7$ 取模。

输入格式

第一行包含整数 n 。

接下来 n 行，每行包含一个整数 a_i 。

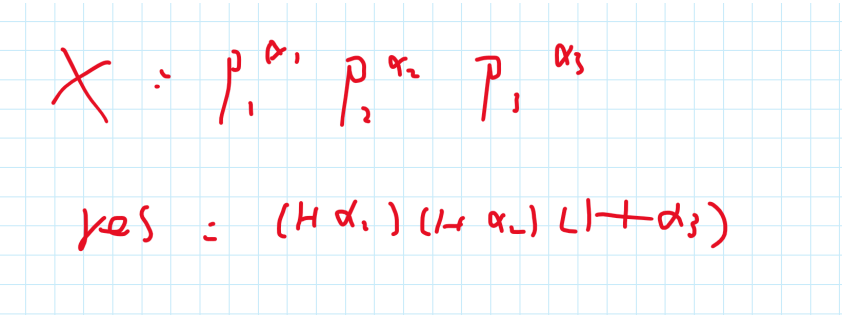
输出格式

输出一个整数，表示所给正整数的乘积的约数个数，答案需对 $10^9 + 7$ 取模。

数据范围

$1 \leq n \leq 100$,
 $1 \leq a_i \leq 2 \times 10^9$

一个数的约数个数通过公式可得是：是他的质因数中(指数+1)的累积



而质因数可以通过因数的质因数求解

```
1 from math import sqrt
2 from collections import Counter
3 n=int(input())
4 # 用于记录指数的累和
5 memo=Counter()
6 mod=10**9+7
7
8 # 求解质因数的模板
9 def cal(x):
10     for i in range(2,int(sqrt(x)+1)):
11         if x%i==0:
12             s=0
13             while x%i==0:
14                 x//=i
15                 s+=1
16             memo[i]+=s
17     if x>1:
18         memo[x]+=1
19
20 for _ in range(n):
21     x=int(input())
```

```
22 # 对每个因数找它的质因数
23 cal(x)
24 nums=[x+1 for x in memo.values()]
25 ans=nums[0]
26 # 计算累乘
27 for i in range(1,len(nums)):
28     ans*=nums[i]
29     ans%=mod
30 print(ans)
```

9 Divisors

给定一个整数 N ，请你求出 $1 \sim N$ 中所有因数个数为 9 的数的个数。 $N \leq 4 \times 10^{12}$

因数个数可以由质因数的冥次求得， $9=1*9=3*3$ ，那么一个数如果满足条件，那么必然是 $x=a^8=a^2b^2$ 的形式

首先找出范围内所有的质因数，质因数的上限一定是 10^6

```
1 n=R()
2
3 ans=0
4
5 MAX=10**6+10
6 # 记录这个数是不是质数，初始为True
7 is_prime=[True]*MAX
8 is_prime[0]=False
9 is_prime[1]=False
10 # 记录质数
11 prime=[]
12 # 对于给定的数据范围，从2开始找质数
13 for i in range(2,MAX):
14     if is_prime[i]:
15         # 如果是质数加入，并把他的冥次标记
16         prime.append(i)
17         # 这里的优化是设置步长并从i**2开始
18         for j in range(i*i,MAX,i):
19             is_prime[j]=False
20
21 i=2
22 while i<n and pow(i,8)<=n:
23     if is_prime[i]:
24         ans+=1
25     i+=1
26
27 # 使用双指针来找
28 p=[v**2 for v in prime]
29 ll=len(p)
30 r=ll-1
31 for i,v in enumerate(p):
32     while i<=r and p[r]*v>n:r-=1
33     if i>=r:break
34     ans+=r-(i+1)+1
```



```
35 | print(ans)
```

约数之和

约数之和同样需要质因数的指数，需要对每个质数计算质数从0到出现次数的累和，然后再累积上

给定 n 个正整数 a_i ，请你输出这些数的乘积的约数之和，答案对 $10^9 + 7$ 取模。

输入格式

第一行包含整数 n 。

接下来 n 行，每行包含一个整数 a_i 。

输出格式

输出一个整数，表示所给正整数的乘积的约数之和，答案需对 $10^9 + 7$ 取模。

数据范围

$1 \leq n \leq 100,$
 $1 \leq a_i \leq 2 \times 10^9$

输入样例：

```
3
2
6
8
```

输出样例：

```
252
```

(2) 约数之和 $(1 + p_1^{a_1} + p_1^{2a_1} + \dots + p_1^{a_1 a_1}) \dots (1 + p_k^{a_k} + p_k^{2a_k} + \dots + p_k^{a_k a_k})$

(3) 约数之和 $(1 + p_1^{a_1} + p_1^{2a_1} + \dots + p_1^{a_1 a_1}) \dots (1 + p_k^{a_k} + p_k^{2a_k} + \dots + p_k^{a_k a_k})$

$$N = p_1^{a_1} \cdot p_2^{a_2} \cdot \dots \cdot p_k^{a_k}$$

```
1 | from math import sqrt
2 | from collections import Counter
3 | n=int(input())
```

```
4 | # 用于记录指数的累和
5 | memo=Counter()
6 | mod=10**9+7
7 |
8 | # 求解质因数的模板
9 | def cal(x):
10 |     for i in range(2,int(sqrt(x)+1)):
11 |         if x%i==0:
12 |             s=0
13 |             while x%i==0:
14 |                 x//=i
15 |                 s+=1
16 |             memo[i]+=s
17 |     if x>1:
18 |         memo[x]+=1
19 |
20 | for _ in range(n):
21 |     x=int(input())
22 |     # 对每个因数找它的质因数，累加指数
23 |     cal(x)
24 |
25 | # 计算每个质数的结果
26 | def count_(k,v):
27 |     ans=0
28 |     for i in range(v+1):
29 |         ans+=pow(k,i)
30 |     return ans
31 | ans=1
32 |
33 | # 计算累和
34 | for k,v in memo.items():
35 |     ans*=count_(k,v)
36 |     ans%=mod
37 | print(ans)
```

欧拉函数

用于求解N在1到N之间有多少个与N互质的数字，需要利用到质因数，这里的特例是1的欧拉函数是1

1 ~ N 中与 N 互质的数的个数被称为欧拉函数，记为 $\phi(N)$ 。

若在算数基本定理中， $N = p_1^{a_1} p_2^{a_2} \dots p_m^{a_m}$ ，则：

$$\phi(N) = N \times \frac{p_1-1}{p_1} \times \frac{p_2-1}{p_2} \times \dots \times \frac{p_m-1}{p_m}$$

质数的欧拉函数是该质数减1

```
1 n=int(input())
2 from math import sqrt
3 # 这里只需要考虑质因数的底数而不需要考虑指数
4 def cal(x):
5     ans=[]
6     for i in range(2,int(sqrt(x)+1)):
7         if x%i==0:
8             s=0
9             while x%i==0:
10                 x//=i
11                 s+=1
12             ans.append(i)
13     if x>1:
14         ans.append(x)
15     return ans
16
17 for _ in range(n):
18     x=int(input())
19     nums=cal(x)
20     ans=x
21     # 利用公式求解
22     for p in nums:
23         ans*=(1-1/p)
24     print(int(ans))
```

求欧拉函数之和

给定一个正整数 n ，求 $1 \sim n$ 中每个数的欧拉函数之和。

输入格式

共一行，包含一个整数 n 。

输出格式

共一行，包含一个整数，表示 $1 \sim n$ 中每个数的欧拉函数之和。

数据范围

$$1 \leq n \leq 10^6$$

输入样例：

6

输出样例：

12

如果对每个数求欧拉函数，时间复杂度为 $O(n\sqrt{n})$ ，而线性筛的方法可以在找质因数的过程中求出 1 到 n 所有数字的欧拉函数。

注意1的欧拉函数为1

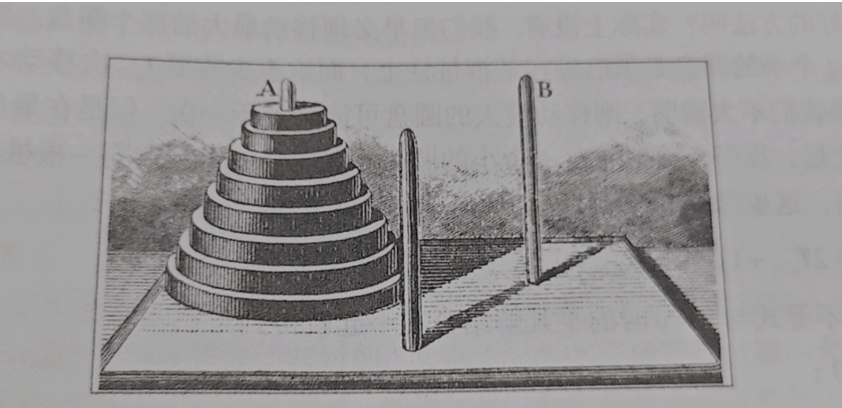
在线性筛中每次枚举 $p \times i$ 的时候顺便求出 $P \times i$ 的欧拉函数，根据公式它的欧拉函数比 i 的多出了 p 乘 $(1 - 1/p)$ ，如果在前面判断了 p 是 i 的因数，就说明这个 p 已经在 i 的欧拉函数中乘过了，欧拉函数和质因数的个数无关和种类有关，因此不需要再乘一次了。

```
1 n=int(input())
2 prime=[]
3 # 做偏移
4 is_prime=[True]*(n+1)
5 res=[0]*(n+1)
6 # 特例
7 res[1]=1
8 # 线性筛
9 for i in range(2,n+1):
10     # 对于一个质数，它的质因数只能是自己，由公式推出欧拉函数是p-1
11     if is_prime[i]:
12         prime.append(i)
13         res[i]=i-1
14     # 注意不能再if语句块中
15     for p in prime:
16         if i*p>n:
17             break
18         is_prime[i*p]=False
19         # 判断
20         if i%p==0:
21             res[i*p]=p*res[i]
22             break
23         else:
24             res[i*p]=res[i]*(p-1)
25 print(sum(res))
26
```

递归

河内塔

如何花费最少的操作次数，在每次只能**移动一个**圆盘并且大圆盘不能**放在小圆盘上**的情形下，将整个圆盘移动到另一个柱子上

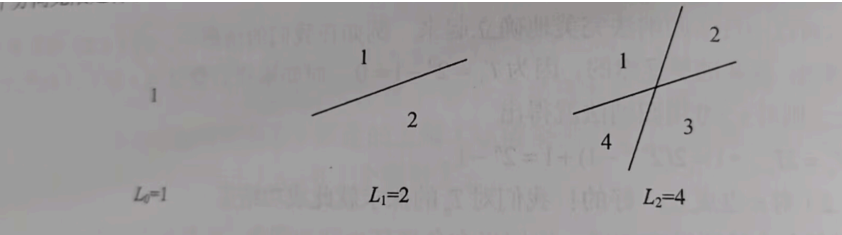


令 T_n 为在规则下将一个 $1\sim n$ 的所有塔移动到另一个柱子上的最少操作次数，一个思路是**先将 $n-1$ 小的圆盘移动到其他柱子上，然后移动第 n 个圆盘，再将 $n-1$ 个移动到第 n 个圆盘上**。则有 $T_n = 2T_{n-1} + 1$

由数学归纳法可得， $T_n = 2^n - 1$

平面上的直线

平面上 n 条直线所能界定的区域的最大个数 L

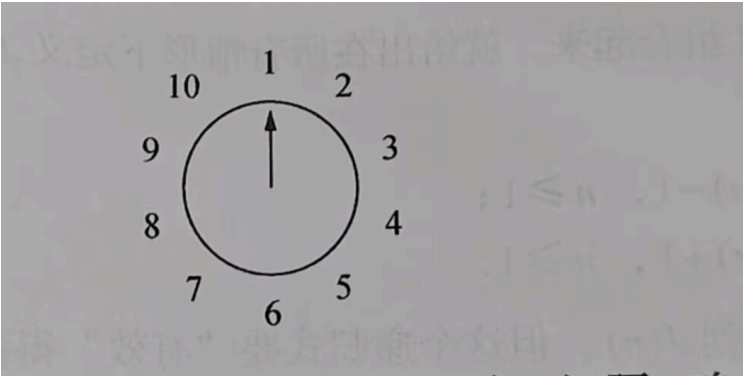


通过观察较小的例子可以假设 $L = 2^n$ ，这是基于每一条线能划分现存的每个区域，但是一条线与 m 条线相交只能划分 $m+1$ 个区域，因此假设 $L_n = L_{n-1} + k$ ，意味着与 $k-1$ 条线相交，而 $k-1 \leq n-1$ ，因此 k 最多等于 n ，则有 $L_n = L_{n-1} + n$

用数学归纳法不断展开有 $L_n = 1 + (1 + n) * n / 2$

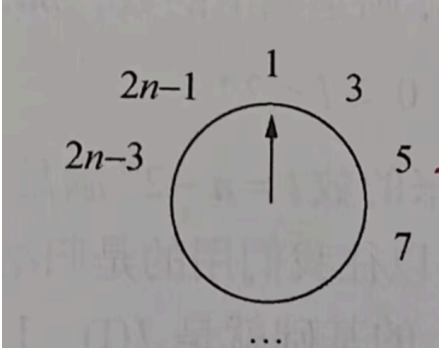
约瑟夫问题

$1\sim n$ 个人围成一个圈，每一个删去一个人，直到最后只剩一个人。



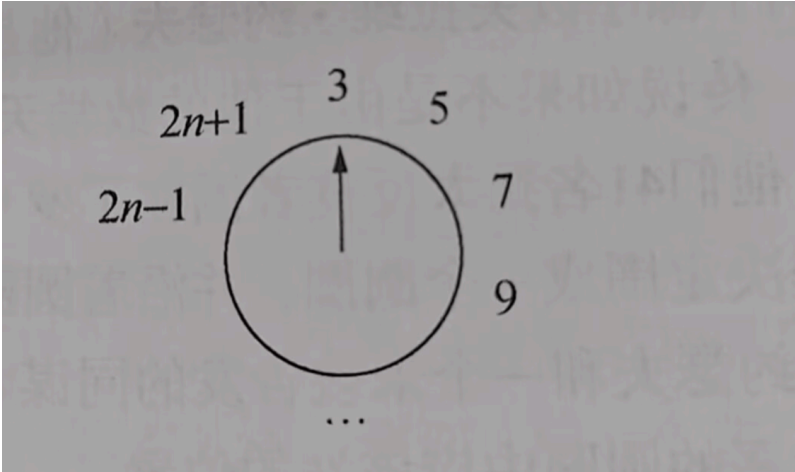
令 $J(n)$ 表示最后剩下的人的编号

假设有 $2n$ 个人，那么经过一轮后有



这种情况可以看作从 n 个人开始，但是编号变为了 $2n-1$ ，那么有 $J(2n)=2*J(n)-1$

对于 $2*n+1$ 个人有相似的情况



$$J(2n+1)=2*J(n)+1$$

则有递归式

$$\begin{aligned} J(1) &= 1; \\ J(2n) &= 2J(n) - 1, \quad n \geq 1; \\ J(2n+1) &= 2J(n) + 1, \quad n \geq 1. \end{aligned}$$

构造几组解，观察有 $J(2^m + l) = 2l - 1$

n	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
$J(n)$	1	1	3	1	3	5	7	1	3	5	7	9	11	13	15	1

从二进制出发

$$\begin{aligned} n &= (1b_{m-1}b_{m-2} \cdots b_1b_0)_2, \\ l &= (0b_{m-1}b_{m-2} \cdots b_1b_0)_2, \\ 2l &= (b_{m-1}b_{m-2} \cdots b_1b_00)_2, \\ 2l+1 &= (b_{m-1}b_{m-2} \cdots b_1b_01)_2, \\ J(n) &= (b_{m-1}b_{m-2} \cdots b_1b_0b_m)_2. \end{aligned}$$

即把n向左循环移动一位，这样不断递归下去就得到了 $J(n) = 2^{cnt} - 1$ ，cnt是n二进制中1的个数

裴属定理

对于不全为0任意整数a和b，对于整数x和y都一定满足ax+by是gcd(a,b)的倍数；特别的一定存在整数 x 和 y 的解，使得 ax+by=gcd(a,b) 成立

一个重要的推论是对于ax+by=1只有在ab互质的情况下有解，反过来也一样如果ab互质，则必须满足ax+by=1

上述的性质可以推广到n个

一个经典运用是：判断是否存在一个ax+by+cy+.....=k(a,b,c...是系数)的问题，这可以用裴属定理，即k是gcd(x,y,z,...)的整数倍则有解

检查「好数组」

给你一个正整数数组 `nums`，你需要从中任选一些子集，然后将子集中每一个数乘以一个 **任意整数**，并求出他们的和。

假如该和结果为 `1`，那么原数组就是一个「好数组」，则返回 `True`；否则请返回 `False`。

示例 1:

输入: `nums = [12,5,7,23]`
输出: `true`
解释: 挑选数字 5 和 7。
 $5*3 + 7*(-2) = 1$

示例 2:

输入: `nums = [29,6,10]`
输出: `true`
解释: 挑选数字 29, 6 和 10。
 $29*1 + 6*(-3) + 10*(-1) = 1$

示例 3:

输入: `nums = [3,6]`
输出: `false`

提示:

- `1 <= nums.length <= 10^5`
- `1 <= nums[i] <= 10^9`

实际上就是裴属定理扩展到n上的性质，即询问是否有

$$x_1 * nums_1 + x_2 * nums_2 + ... + x_k * nums_k = 1$$

那么直接判断数组中的数是否互质，求解最大公约数是否等于1

欧几里得算法

求出所有的公约数

a和b所有的公约数是a和b的最大公约数的约数。

```
1 temp=set()
2 n=gcd(a,b)
3 i=1
4 while i<=int(n**0.5)+1:
5     if not n%i:
6         temp.add(i)
7         # 注意另一部分不能落下
8         temp.add(n//i)
9     i+=1
10 temp=sorted(temp)
```

基本欧几里得算法

```
1 # 迭代的写法
2 def gcd(x,y):
3     if x<y:
4         x,y=y,x
5     m=-1
6     while m:
7         _,m=divmod(x,y)
8         x=y
9         y=m
10        temp=m
11    return m
12
13 # 递归的写法，根据性质：gcd(a,b)=gcd(b,a%b)
14 def gcd(a,b):
15     if not b:
16         return a
17     return gcd(b,a%b)
18
```

扩展欧几里得算法

但是仅仅根据欧几里得算法无法得出系数，需要用到扩展的欧几里得算法，即在找最大公约数的过程中求解出系数。

存在 x,y 使得 $ax+by=\gcd(a,b)$

当 $b=0$ 时, $ax+by=a$, 易得 $x=1,y=$ 任何数

当 $b\neq 0$ 时,

因为

$$ax+by=\gcd(a,b)=\gcd(b,a\bmod b)$$

而

$$bx'+(a\bmod b)y'=\gcd(b,a\bmod b)$$

$$bx'+(a-\left\lfloor\frac{a}{b}\right\rfloor b)y'=\gcd(a,b)$$

$$ay'+b(x'-\left\lfloor\frac{a}{b}\right\rfloor y')=ax+by$$

将 x' 和 y' 位置互换

$$ax'+b(y'-\left\lfloor\frac{a}{b}\right\rfloor x')=ax+by$$

故

$$x=x',y=y'-\left\lfloor\frac{a}{b}\right\rfloor x'$$

在调用欧几里得算法的过程中，反转了a和b，因此x和y传入的时候也要反转。根据推到可以得到外层的系数和下一层循环的系数右一定的联系。

给定 n 对正整数 a_i,b_i , 对于每对数, 求出一组 x_i,y_i , 使其满足 $a_i\times x_i+b_i\times y_i=\gcd(a_i,b_i)$ 。

输入格式

第一行包含整数 n 。

接下来 n 行, 每行包含两个整数 a_i,b_i 。

输出格式

输出共 n 行, 对于每组 a_i,b_i , 求出一组满足条件的 x_i,y_i , 每组结果占一行。

本题答案不唯一, 输出任意满足条件的 x_i,y_i 均可。

数据范围

$1\leq n\leq 10^5,$
 $1\leq a_i,b_i\leq 2\times 10^9$

输入样例：

```
2
4 6
8 18
```

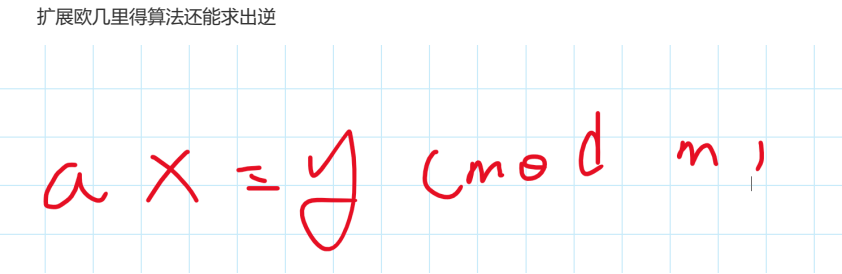
输出样例：

```
-1 1
-2 1
```

注意调用扩展欧几里得时，后两个传入0, 0

```
1 def exgcd(a,b,x,y):
2     if not b:
3         # 如果b等于0, 根据公式一定有ax+by=a, 选择返回系数1, 0
4         return a,1,0
5     # 反之, 递归
6     d,y,x=exgcd(b,a%b,y,x)
7     return d,x,y-a//b*x
8 t=int(input())
9 for _ in range(t):
10    x,y=[int(x) for x in input().split()]
11    # 初始的参数都为0
12    _,a,c=exgcd(x,y,0,0)
13    print(*[a,c])
```

扩展欧几里得求逆元



当ab互素时, 即ax+by==1, 求解出的x是a模b的逆元, y是b模a的逆元。

线性同余方程

给定 n 组数据 a_i, b_i, m_i , 对于每组数求出一个 x_i , 使其满足 $a_i \times x_i \equiv b_i \pmod{m_i}$, 如果无解则输出 `impossible`。

输入格式

第一行包含整数 n 。

接下来 n 行, 每行包含一组数据 a_i, b_i, m_i 。

输出格式

输出共 n 行, 每组数据输出一个整数表示一个满足条件的 x_i , 如果无解则输出 `impossible`。

每组数据结果占一行, 结果可能不唯一, 输出任意一个满足条件的结果均可。

输出答案必须在 `int` 范围之内。

数据范围

$1 \leq n \leq 10^5$,
 $1 \leq a_i, b_i, m_i \leq 2 \times 10^9$

输入样例:

```
2
2 3 6
4 3 5
```

输出样例:

```
impossible
-3
```

找a和m的最大公约数, 如果这个数可以被b整除则存在x, 结果是 `b//d*(a的系数)%m`

```
1 t=int(input())
2 def exgcd(a,b,x,y):
3     if not b:
4         return a ,1,0
5     d,y,x=exgcd(b,a%b,y,x)
6     return d,x,y-a//b*x
7
8 for _ in range(t):
9     a,b,m=[int(x) for x in input().split()]
10    d,x,y=exgcd(a,m,0,0)
11    if b%d:
12        print('impossible')
13    else:
14        print((x*b//d)%m)
```

逆元

就是扩展欧几里得算法中求解出的x或者是y的系数, 另外根据费马小定理有: **当取模的数是质数的时候, a的逆元就是a^m-1**。在对两个较大的数相除的结果取模的时候, 由于同余定理对除法不起作用, 因此要把被除数转化为逆元变为乘法后取模, 而找一个数取逆元的过程如下:

$$b^{m-1} \equiv 1 \mod m$$

和下面这条

$$a/b \equiv a * x \mod m$$

,就可以联立推出

$$a/b * b^{m-1} \equiv a * x \mod m$$

,所以得到

$$x \equiv b^{m-2} \mod m$$

这里可以使用费马小定理主要是因为10**9+7是一个质数

排列

对于长度为n的数组它的全排列个数为n!, 如果数组中有重复的元素那么总的排列个数要除以数组中相同元素的个数的阶乘。

第二类斯特林数

第二类斯特林数S(n,m)表示的是把n个不同的小球放在m个相同的盒子里方案数，可以递推求解：一个球单独放f(n-1,m-1)以及放入前面的盒子中f(n-1,m)*m

计算分配糖果的不同方式

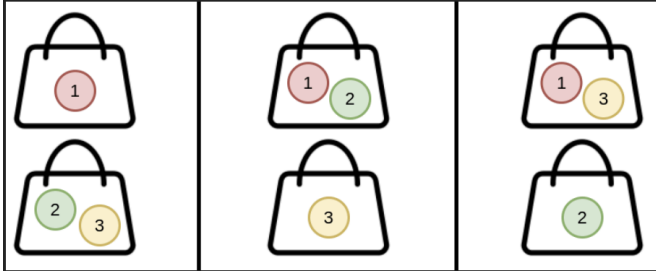
现有 n 颗 不同 糖果（分别标记为 1 到 n）和 k 个相同的手袋。请把糖果分配到各个手袋中并保证每个手袋里至少有一颗糖果。

不考虑手袋和糖果的摆放顺序，会有多种不同的分配方式。如果某种分配方式中其中一个手袋里的糖果与另一种分配方式中所有手袋里的糖果都不相同，则认为这两种分配方式不同。

例如，(1)，(2,3) 与 (2)，(1,3) 的分配方式是不同的，因为第一种分配方式中手袋(2,3)里的糖果2和3，在第二种分配方式中被分配到了手袋 (2) 和 (1,3) 中。

已知整数 n 和 k，请返回分配糖果的不同方式。返回的答案如果数值太大，请取10⁹ + 7的模，并返回。

示例 1:



输入: n = 3, k = 2
输出: 3
解释: 把糖果 3 分配到 2 个手袋中的一个，共有 3 种方式:
(1), (2,3)
(1,2), (3)
(1,3), (2)

提示:

- 1 <= k <= n <= 1000

第二类斯特林数，要求每个盒子至少有一个球。

```
1 class Solution:
2     def waysToDistribute(self, n: int, k: int) -> int:
3         mod=10**9+7
4         f=[[0]*(n+1) for _ in range(k+1)]
5         # 0个盒子0个球的时候有一种可能，如果球数更多则不满足要求(都要放到盒子中)
6         f[0][0]=1
7         for i in range(1,k+1):
8             # 球的个数需要大于等于盒子数
9             for j in range(i,n+1):
10                 f[i][j]=(f[i-1][j-1]+f[i][j-1]*i)%mod
11         return f[k][n]
```

组合数

递推式

组合数的递推公式为： $C_a^b = C_{a-1}^b + C_{a-1}^{b-1}$ 注意是加法

注意组合数在python3中可以直接用comb快速计算，但是在pypy3中并没有comb函数，即使comb对于大数很快但是对于多次询问还是需要打表求解。

对于长度为k的数组任意取有2^k-1个非空序列

给定 n 组询问，每组询问给定两个整数 a, b ，请你输出 $C_a^b \bmod (10^9 + 7)$ 的值。

输入格式

第一行包含整数 n 。

接下来 n 行，每行包含一组 a 和 b 。

输出格式

共 n 行，每行输出一个询问的解。

数据范围

$1 \leq n \leq 10000,$
 $1 \leq b \leq a \leq 2000$

输入样例：

```
3
3 1
5 3
2 2
```

输出样例：

```
3
10
1
```

对于多次询问（十万次）必须快速求解出组合数，因此选择打表计算对于给定的数据打表最多是10**6。

初始化dp[0][0]=1

```
1 dp=[ [0]*2010 for _ in range(2010)]
2 mod=10**9+7
3 for i in range(2010):
4     # 内层的循环只能到i，在往外无意义
5     for j in range(i+1):
6         # j=0的时候结果为1
7         if not j:dp[i][j]=1
8         else:
9             dp[i][j]=(dp[i-1][j-1]+dp[i-1][j])%mod
10 t=int(input())
11 for _ in range(t):
12     a,b=[int(x) for x in input().split()]
13     if b>a:
14         a,b=b,a
15     print(dp[a][b])
16
```

阶乘

给定 n 组询问，每组询问给定两个整数 a, b ，请你输出 $C_a^b \bmod (10^9 + 7)$ 的值。

输入格式

第一行包含整数 n 。

接下来 n 行，每行包含一组 a 和 b 。

输出格式

共 n 行，每行输出一个询问的解。

数据范围

$1 \leq n \leq 10000,$
 $1 \leq b \leq a \leq 10^5$

这里给定的数据范围无法打表计算每一组解，根据组合数的性质求解解除使用递推求解每个数的阶乘，然后在计算中使用阶乘求解即可。


```
1 n=10**5+10
2 mod=10**9+7
3 dp=[1]*n
4 infact=[0]*n
5 for i in range(1,n):
6     dp[i]=(i*dp[i-1])%mod
7 t=int(input())
8 for _ in range(t):
9     a,b=[int(x) for x in input().split()]
10    # 转换为逆元进行乘法操作, 才能取模
11    print(dp[a]*pow(dp[b],mod-2,mod) *pow(dp[a-b],mod-2,mod)%mod)
```

Lucas

给定 n 组询问, 每组询问给定三个整数 a, b, p , 其中 p 是质数, 请你输出 $C_a^b \bmod p$ 的值。

输入格式

第一行包含整数 n 。

接下来 n 行, 每行包含一组 a, b, p 。

输出格式

共 n 行, 每行输出一个询问的解。

数据范围

$1 \leq n \leq 20,$
 $1 \leq b \leq a \leq 10^{18},$
 $1 \leq p \leq 10^5,$

对于数据范围很大的组合数取模的结果, 如果模数是一个不大的质数那么可以用卢卡斯定理求解:

Lucas 定理内容如下: 对于质数 p , 有

$$\binom{n}{m} \bmod p = \binom{\lfloor n/p \rfloor}{\lfloor m/p \rfloor} \cdot \binom{n \bmod p}{m \bmod p} \bmod p$$

```
1 def C(a,b,p):
2     j=a
3     res=1
4     # 根据定义计算, 根据公式拆分为(a/b)*(c/d)的形式..., 一共有b项, 这个过程中一直取模
5     for i in range(1,b+1):
6         res=res*j%p
7         # 使用逆元
8         res=res*pow(i,p-2,p)%p
9         j-=1
10    return res
11
12 def locus(a,b,p):
13    # 如果ab的范围可以接收(十万以内), 直接计算
```

```
14 if a<p and b<p:
15     return C(a,b,p)
16 # 否则用卢卡斯定理简化计算
17 return C(a%p,b%p,p)*locus(a//p,b//p,p)%p
18 for _ in range(int(input())):
19     a,b,p=[int(x) for x in input().split()]
20     print(locus(a,b,p))
```

中国剩余定理

中国剩余定理是扩展lucas定理, 可以求解lucas中P不为质数的情况。

「物不知数」问题: 有物不知其数, 三三数之剩二, 五五数之剩三, 七七数之剩二。问物几何?

中国剩余定理 (Chinese Remainder Theorem, CRT) 可求解如下形式的一元线性同余方程组 (其中 $n_1, n_2, \dots, n_k$ 两两互质) :

$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ x \equiv a_2 \pmod{n_2} \\ \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$

其中a和m是已知的, 注意m是互质(最大公约数为1)的要求, 求解x。

- 计算所有模数的积 n ;
- 对于第 i 个方程:
 - 计算 $m_i = \frac{n}{n_i}$;
 - 计算 m_i 在模 n_i 意义下的逆元 m_i^{-1} ;
 - 计算 $c_i = m_i m_i^{-1}$ (不要对 n_i 取模)。
- 方程组在模 n 意义下的唯一解为: $x = \sum_{i=1}^k a_i c_i \pmod{n}$ 。

反过来可以用于求解大数取模合数时的结果, 例如, 其中a1.a2.a3...是comb(n,k)取模2,3,4679的结果, 只能运用于冥次为1的情况

现在考虑如何计算：

$$\sum_{k|n} \binom{n}{k} \bmod 999\,911\,658$$

因为 999 911 658 不是质数，无法保证 $\forall x \in [1, 999\,911\,657], x$ 都有逆元存在，上面这个式子我们无法直接计算。

注意到 $999\,911\,658 = 2 \times 3 \times 4679 \times 35617$ ，其中每个质因子的最高次数均为一，我们可以考虑分别求出 $\sum_{k|n} \binom{n}{k}$ 在模 2, 3, 4679, 35617 这几个质数下的结果，最后用中国剩余定理来合并答案。

也就是说，我们实际上要求下面一个线性方程组的解：

$$\begin{cases} x \equiv a_1 \pmod{2} \\ x \equiv a_2 \pmod{3} \\ x \equiv a_3 \pmod{4679} \\ x \equiv a_4 \pmod{35617} \end{cases}$$

而计算一个组合数对较小的质数取模后的结果，可以利用 [卢卡斯定理](#)。

判断操作后字符串中的数字是否相等 II

给你一个由数字组成的字符串 `s`。重复执行以下操作，直到字符串恰好包含 **两个** 数字：

- 从第一个数字开始，对于 `s` 中的每一对连续数字，计算这两个数字的和 模 10。
- 用计算得到的新数字依次替换 `s` 的每一个字符，并保持原本的顺序。

如果 `s` 最后剩下的两个数字相同，则返回 `true`。否则，返回 `false`。

示例 1：

输入：s = "3902"

输出：true

解释：

- 一开始，s = "3902"
- 第一次操作：
 - `(s[0] + s[1]) % 10 = (3 + 9) % 10 = 2`
 - `(s[1] + s[2]) % 10 = (9 + 0) % 10 = 9`
 - `(s[2] + s[3]) % 10 = (0 + 2) % 10 = 2`
 - s 变为 "292"
- 第二次操作：
 - `(s[0] + s[1]) % 10 = (2 + 9) % 10 = 1`
 - `(s[1] + s[2]) % 10 = (9 + 2) % 10 = 1`
 - s 变为 "11"
- 由于 "11" 中的数字相同，输出为 true。

提示：

- `3 <= s.length <= 105`
- s 仅由数字组成。

最后化简出的两个数实际上就是s的n-1位(前n-1位以及后n-1位)与杨辉三角形的第n-1层——相乘，关键在于如何快速求解出杨辉三角形的n-1层的系数。

根据公式杨辉三角形的第n层的第m个，数值为C(n-1,m-1)由数据范围可知无法递推求解，但是通过阶乘或者lacus定理需要模数为质数，然而这里给的是模10。因此要使用扩展lucas定理使得可以模一个合数。

将10拆为2和5然后运用公式

$$\binom{n}{k} \bmod 10 = (5r_2 + 6r_5) \bmod 10$$

将10拆位2和5构造中国剩余定理的公式有：
 $a1 = r1 = \binom{n}{k} \bmod 2$ $a2 = r2 = \binom{n}{k} \bmod 5$, 而 $n1=5,n2=2$, 由此可以构造公式求解 x 即
 $\binom{n}{k} \bmod 10$

```
1  MX=5
2  C=[[0]*MX for _ in range(MX)]
3  for i in range(MX):
4      for j in range(i+1):
5          if not j:C[i][j]=1
6          else:C[i][j]=C[i-1][j-1]+C[i-1][j]
7
8  class Solution:
9      def hasSameDigits(self, s: str) -> bool:
10         n=len(s)-1
11         mod=10
12         # lucas定理
13         def lucas(n,k,p):
14             # 第二个数为0
15             if not k:return 1
16             # 数值范围较小可以直接求解
17             return C[n%p][k%p]*lucas(n//p,k//p,p)%p
18
19         def cal(n,k):
20             # 经过变换后的公式
21             return (5*lucas(n,k,2)+6*lucas(n,k,5))%10
22
23         temp1=temp2=idx=0
24         s=[int(c) for c in s]
25         # 枚举两个位置
26         for i,j in pairwise(s):
27             val=cal(n-1,idx)
28             temp1+=val*i
29             temp2+=val*j
30             idx+=1
31         # 比较
32         return temp1%10==temp2%10
33
34
35
```

卡特兰数

一种特殊的组合数

问题一般是两个选择，问这两种选择满足的组合有多少种

(2)求01串的个数：n个0与n个1构成的序列方案数，使得任何一个前缀0的个数不少于1的个数；

Solution：将0看做在坐标系中向右走一步，1看做向上走一步，则问题可简化为从原点到(n,n)所有路线中一直处于y=x之下（不越过直线y=x）的不同路径方案数，与上题相同，方案数即为对应n的卡特兰数；

(3)给定节点求解二叉树的个数：已知由n个节点，求形成不同的二叉树有多少种？

Solution：将向左生成子树看做0，向右生成子树看做1，则问题简化为求n/2个0和n/2个1构成数串的不同方案数，即上题无条件解的一半，与上题答案相同，形成不同的树的个数即为对应n的卡特兰数；

(4)求凸边形进行三角剖分的不同方案数：在一个有n+3条边的凸多边形中，求通过若干条互不相交的对角线，把这个多边形划分成若干个三角形的不同方案数。

Solution：因为每一条边都一定是剖分后的三角形的一条边，任意一条边都会把多边形分成两个小多边形，那么根据乘法原理，解即为划分成不同多边形的方案数对应小多边形的划分方案数之和，即 $f[n]=\sum(3\leq k\leq n-3)f[k]f[n-k-1]$ ，可以发现解数即为n对应的卡特兰数；

(5)n对括号正确匹配数目:给定n对括号，求括号正确配对的字符串数；

Solution：因为是匹配问题，那么最后一个左括号必然有唯一右括号与其匹配，假设f[n]为n对括号的正确配对数目，那么有递推关系 $f[n]=f[0]f[n-1]+f[1]f[n-2]+...+f[n-1]f[0]$ ，显然f[n]是n对应的卡特兰数。

生成字符串

lxxhgw 最近接到了一个生成字符串的任务，任务需要他把 n 个 1 和 m 个 0 组成字符串，但是任务还要求在组成的字符串中，在任意的 k 个字符中，1 的个数不能少于 0 的个数。现在 lxxhgw 想要知道满足要求的字符串共有多少个，聪明的程序员们，你们能帮助他吗？

输入格式

输入数据只有一行，包括 2 个数字 n 和 m 。

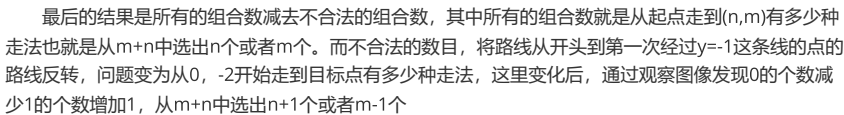
输出格式

输出数据是一行，包括 1 个数字，表示满足要求的字符串数目，这个数可能会很大，只需输出这个数除以 20100403 的余数

输入输出样例

输入 #1	输出 #1
2 2	2

把文图抽象为图像，1相当于向右上走，0相当于向右下走，为了1的个数要大于0需要在组合的过程中不要经过y=-1这条线否则就是0的个数大于1的个数



鸡蛋饼

4057031

矩阵ii

64/103

```
1 f=[0]*110
2 mod=100
3 f[0]=1
4 for i in range(1,101):
5     # 从0到n-1
6     for j in range(i):
7         f[i]=(f[i]+f[j]*f[i-1-j])%mod
8 n=int(input())
9 print(f[n])
```

快速冥

给定 n 组 a_i, b_i, p_i , 对于每组数据, 求出 $a_i^{b_i} \bmod p_i$ 的值。

输入格式

第一行包含整数 n 。

接下来 n 行, 每行包含三个整数 a_i, b_i, p_i 。

输出格式

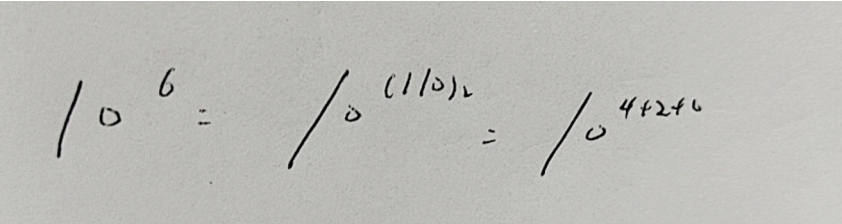
对于每组数据, 输出一个结果, 表示 $a_i^{b_i} \bmod p_i$ 的值。

每个结果占一行。

数据范围

$1 \leq n \leq 100000,$
 $1 \leq a_i, b_i, p_i \leq 2 \times 10^9$

对于较大的冥次比如说n达到10**9, 保证不会溢出的情况下使用迭代求解会超时。快速冥的做法是把冥次拆分为二进制数 (比如6写为110, $2^2 + 2^1$), 然后只操作这些位



这样就只需要迭代log(n)次

```
1 n=int(input())
2 def cal(a,b,p):
3     res=1
4
5     while b:
6         # 只有在有1的时候才会葛更新结果, 也就是每次乘的时候恰好对应到拆分的位数
7         if b&1:
8             res=(res*a)%p
```

```
9         b=b>>1
10        a=(a*a)%p
11        return res
12
13 for _ in range(n):
14     a,b,p=[int(x) for x in input().split()]
15     print(cal(a,b,p))
```

如何快速计算 x^8 ?

$$x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8$$

从 x 开始，不断地对自身**平方**，
重复三次即可得到 x^8 。

如何快速计算 x^{15} ?

$$x \cdot x^2 \cdot x^4 \cdot x^8 = x^{15}$$

在 $x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8$ 的过程中，
把这些数全部相乘，即可得到 x^{15} 。

如何快速计算 x^{13} ?

$$x \cdot x^4 \cdot x^8 = x^{13}$$

在 $x \rightarrow x^2 \rightarrow x^4 \rightarrow x^8$ 的过程中，
把 x, x^4, x^8 相乘，即可得到 x^{13} 。

如何知道要把哪些数相乘？

$$13 = 1101_{(2)}$$

把 n 看成**二进制数**，
从低到高遍历二进制数，
遇到 1 就和 x 的幂相乘。

如何遍历二进制数中的 1?

$$1101 \rightarrow 110 \rightarrow 11 \rightarrow 1$$

不断**右移**二进制数，
右移前看**最低位**是否为 1。

如何处理 $n < 0$ 的情况？

$$x^{-n} = \left(\frac{1}{x}\right)^n$$

把 n 变成 $-n$ ，把 x 变成 $1/x$ ，
就转换成了 $n \geq 0$ 的情况。

冥次

使用内置函数Pow可快速求解，为了防止数据太大可以在**Pow函数中加入第三个参数**作为取模的值防止过大导致计算较慢。

内置函数sqrt()可以求解开方，isqrt()可以求出开方后的整数。

补充2^31开方后的结果为40000多

比起log函数求冥，模拟整除更好

曼哈顿距离转换为切比雪夫距离

将所有点按照原点顺时针旋转45°再乘根号2倍转换到切比雪夫坐标系

最小化曼哈顿距离

给你一个下标从 0 开始的数组 points，它表示二维平面上一些点的整数坐标，其中 points[i] = [xi, yi]。

两点之间的距离定义为它们的**曼哈顿距离**。

请你恰好移除一个点，返回移除后任意两点之间的**最大**距离可能的**最小**值。

示例 1:

输入: points = [[3,10],[5,15],[10,2],[4,4]]

输出: 12

解释: 移除每个点后的最大距离如下所示:

- 移除第 0 个点后，最大距离在点 (5, 15) 和 (10, 2) 之间，为 |5 - 10| + |15 - 2| = 18。
 - 移除第 1 个点后，最大距离在点 (3, 10) 和 (10, 2) 之间，为 |3 - 10| + |10 - 2| = 15。
 - 移除第 2 个点后，最大距离在点 (5, 15) 和 (4, 4) 之间，为 |5 - 4| + |15 - 4| = 12。
 - 移除第 3 个点后，最大距离在点 (5, 15) 和 (10, 2) 之间的，为 |5 - 10| + |15 - 2| = 18。
- 在恰好移除一个点后，任意两点之间的最大距离可能的最小值是 12。

示例 2:

输入: points = [[1,1],[1,1],[1,1]]

输出: 0

解释: 移除任一点后，任意两点之间的最大距离都是 0。

提示:

- 3 <= points.length <= 10³
- points[i].length == 2
- 1 <= points[i][0], points[i][1] <= 10⁸

对于给定的数据范围无法得到最大的曼哈顿距离（因为要两层循环），同时无法枚举删除每一个点后的最大距离。但是可以通过将曼哈顿距离转化为切比雪夫距离一次遍历求出最大的曼哈顿距离。

切比雪夫距离（Chebyshev distance）是向量空间中的一种度量，二个点之间的距离定义为其各坐标数值差的最大值。¹

在二维空间内，两个点之间的切比雪夫距离为它们横坐标之差的绝对值与纵坐标之差的绝对值的最大值。设点 $A(x_1, y_1), B(x_2, y_2)$ ，则 A, B 之间的切比雪夫距离用公式可以表示为：

$$d(A, B) = \max(|x_1 - x_2|, |y_1 - y_2|)$$

n 维空间中切比雪夫距离的距离公式可以表示为：

$$\begin{aligned} d(x, y) &= \max \{|x_1 - y_1|, |x_2 - y_2|, \dots, |x_n - y_n|\} \\ &= \max \{|x_i - y_i|\} (i \in [1, n]) \end{aligned}$$

所以，曼哈顿距离与切比雪夫距离之间会不会有联系呢？

接下来我们简略证明一下：

假设 $A(x_1, y_1), B(x_2, y_2)$,

我们把曼哈顿距离中的绝对值拆开，能够得到四个值，这四个值中的最大值是两个非负数之和，即曼哈顿距离。则 A, B 两点的曼哈顿距离为：

$$\begin{aligned} d(A, B) &= |x_1 - x_2| + |y_1 - y_2| \\ &= \max \{x_1 - x_2 + y_1 - y_2, x_1 - x_2 + y_2 - y_1, x_2 - x_1 + y_1 - y_2, x_2 - x_1 + y_2 - y_1\} \\ &= \max(|(x_1 + y_1) - (x_2 + y_2)|, |(x_1 - y_1) - (x_2 - y_2)|) \end{aligned}$$

我们很容易发现，这就是 $(x_1 + y_1, x_1 - y_1), (x_2 + y_2, x_2 - y_2)$ 两点之间的切比雪夫距离。

在原坐标系中通过将点的横纵坐标转换为 $(x+y, y-x)$ 可以转化到切比雪夫坐标系中，把所有点转化到切比雪夫坐标系中可以求解切比雪夫距离这个距离等价于原坐标系中的曼哈顿距离，而切比雪夫距离通过最大与最小的横/纵的相减就可以求出，也就是一次遍历把所有点换到切比雪夫中同时维护值的大小，之后枚举删除每一个值后的结果。

```
1 from sortedcontainers import SortedList as sl
2 class Solution:
3     def minimumDistance(self, points: List[List[int]]) -> int:
4         # 使用有序集合维护
5         x=sl()
6         y=sl()
7         for l,r in points:
8             x.add(l+r)
9             y.add(r-l)
10        ans=inf
11        for l,r in points:
12            # 先删除在加入
13            x.remove(l+r)
14            y.remove(r-l)
15            ans=min(ans,max(x[-1]-x[0],y[-1]-y[0]))
```

```
16         x.add(l+r)
17         y.add(r-l)
18         return ans
```

容斥原理

在计算结果的时候不去考虑重叠的部分，先把所有的加上然后再家去重叠的部分，这个公式就是概率论中的几个集合相交

$$\begin{aligned} |S_1 \cup S_2 \cup S_3| &= |S_1| + |S_2| + |S_3| - |S_1 \cap S_2| - |S_1 \cap S_3| - |S_2 \cap S_3| \\ &\quad + |S_1 \cap S_2 \cap S_3| \end{aligned}$$

容斥原理

往外推广的结果是 $1-2+3-4+5-6+\dots+(-1)^{(n-1)}*n$ ，奇数个集合是加偶数个集合是减。

$$C_n^1 + C_n^2 + C_n^3 + C_n^4 + \dots + C_n^n = 2^n - 1$$

可以看作是从 n 个数中选取任意个，一共有 2^n-1 个状态（注意不会选0个，等式左边会补上 C_n^0 ）

给定一个整数 n 和 m 个不同的质数 $p_1, p_2, \dots, p_m$ 。

请你求出 $1 \sim n$ 中能被 $p_1, p_2, \dots, p_m$ 中的至少一个数整除的整数有多少个。

输入格式

第一行包含整数 n 和 m 。

第二行包含 m 个质数。

输出格式

输出一个整数，表示满足条件的整数的个数。

数据范围

$1 \leq m \leq 16,$
 $1 \leq n, p_i \leq 10^9$

输入样例：

```
10 2
2 3
```

输出样例：

```
7
```

使用容斥原理找能被一个质数整除的个数，这里补充一点1到n中能被p整除的个数是n//p，而能被n和m同时整除的个数是n//(m*n)，这里的p是质数。

能被质数整除的集合一共有m个，枚举每个选或者是不选则有2^m-1个状态

```
1 n,m=[int(x) for x in input().split()]
2 prime=[int(x) for x in input().split()]
3 res=0
4 # 使用位图表示当前位置的集合有没有选
5 for i in range(1,1<<len(prime)):
6     # 记录选取的集合的乘积
7     t=1
8     # 分别记录当前枚举的集合个数和结果
9     cnt=summ=0
10    # 遍历每一位
11    for j in range(m):
12        p=prime[j]
13        if i>>j&1:
14            t*=p
15            # 如果累乘的结果大于N说明不存在能被当前集合所除的数
16            if t>n:
17                t=-1
18                break
19            cnt+=1
20    summ+=n//t
21    if t!=-1:
22        # 奇数加偶数减
23        res+= summ if cnt&1 else -summ
24 print(res)
```

单面值组合的第 K 小金额

给你一个整数数组 `coins` 表示不同面额的硬币，另给你一个整数 `k`。

你有无限量的每种面额的硬币。但是，你 **不能** 组合使用不同面额的硬币。

返回使用这些硬币能制造的 第 `kth` 小 金额。

示例 1：

输入：coins = [3,6,9], k = 3

输出：9

解释：给定的硬币可以制造以下金额：
3元硬币产生3的倍数：3, 6, 9, 12, 15等。
6元硬币产生6的倍数：6, 12, 18, 24等。
9元硬币产生9的倍数：9, 18, 27, 36等。
所有硬币合起来可以产生：3, 6, 9, 12, 15等。

示例 2：

输入：coins = [5,2], k = 7

输出：12

解释：给定的硬币可以制造以下金额：
5元硬币产生5的倍数：5, 10, 15, 20等。
2元硬币产生2的倍数：2, 4, 6, 8, 10, 12等。
所有硬币合起来可以产生：2, 4, 5, 6, 8, 10, 12, 14, 15等。

二分枚举金额的大小，查看是否满足至少有k个小于等于该金额

注意这里的金额不能有重复值，也就是不能直接取整除后的结果，需要减去重复的值，不仅两个值之间要减去重复的内容，三个值四个值之间也同样需要，因此使用容斥定理求解。

容斥定理枚举子集找出子集的最小公倍数，然后选择减或加。

```
1 class Solution:
2     def findKthSmallest(self, coins: List[int], k: int) -> int:
3         l,r=0,min(coins)*k
4         n=len(coins)
5         memo={}
6         # 预处理出每个子集的最小公倍数
7         for s in range(1,1<<n):
8             lcm=1
9             for j in range(n):
10                 if s>>j&1:lcm=math.lcm(lcm,coins[j])
11             memo[s]=lcm
12
13         def cal(x):
14             cnt=0
```



```
15         for i in range(1,1<=<n):
16             c=0
17             lcm=1
18             # 根据子集中元素的个数判断是加还是减(奇数加偶数减)
19             c=i.bit_count()
20             lcm=memo[i]
21             cnt+=pow(-1,c+1)*(x//lcm)
22         return cnt>=k
23
24     while l<=r:
25         mid=(l+r)>>1
26         if cal(mid):
27             r=mid-1
28         else:
29             l=mid+1
30     return r+1
```

简单博弈论

NIM游戏

n堆物品，每堆有ai个，两个玩家轮流取走任意一堆的任意个物品，但不能不取。取走最后一个物品的人获胜

结论是每堆石子的异或和如果等于0，则先手必输反之必胜。

Training Before the Olympiad

有一个正整数序列a，两人进行游戏，规则如下：

- 当序列中只有一个元素时，游戏结束。
- 否则，两人交替操作，每次需要选出a中的两个元素 $a_i,a_j(i\neq j)$ ，将它们删除并插入一个值为 $\lfloor \frac{a_i+a_j}{2} \rfloor \cdot 2$ 的元素。

先手希望使得最终a中留下的数最大化，后手希望最小化，两人都按最优策略进行游戏，求对于给定的长为n的序列a的每个非空前缀，对该前缀进行游戏的结果，即最后留下的数。每个测试点t组测试用例。

先手想要结果最大后手想要结果最小，每次选奇偶不同的数可以使得元素总和减小一，而相同的总和不变，因此前者尽可能多的选奇偶相同的后者选不同的，此外为了保证先手胜利，先手会每次优先奇数/偶数优先保证后手不能减小总和。

维护两个前缀数组，一个是奇数的个数一个是前缀和，每一轮如果奇数的个数足够的话可以减少三个如果不够，剩余0个或2个那么最后的总和一定不变，反之一定会减小1.

```
1 import sys
2 from itertools import accumulate as ac
3 input=sys.stdin.readline
4
5
6 for _ in range(int(input())):
7     n=int(input())
8     nums=list(map(int,input().split()))
9     # 记录奇数个数
10    odd=[0]*n
11    odd[0]=int(nums[0]&1==1)
12    for i in range(1,n):
13        odd[i]+=odd[i-1]+int(nums[i]&1==1)
14    # 记录前缀和
15    pre=list(ac(nums))
16    ans=[]
17    for i in range(n):
18        # 对于第一个要特判
19        if i==0:
20            ans.append(pre[i])
21            continue
22        v=odd[i]
23        a,b =divmod(v,3)
24        # 总要和减去轮数
25        if b==0 or b==2:
26            ans.append(pre[i]-a)
27        else:
28            ans.append(pre[i]-a-1)
29    for v in ans:
30        print(v,end=' ')
31    print()
```

树状数组

类似于线段树可以快速求出区间和，同时可以更新，这两个操作的时间复杂度为logn,不过这里的更新只能是单点修改。

树状数组的关键在于巧妙地区间拆分

启示：如果把一个正整数 i 拆分成若干个不同的 2 的幂（从大到小），那么只会拆分出 $\mathcal{O}(\log i)$ 个数。前缀能否也这样拆分呢？

举个例子， $13 = 8 + 4 + 1$ ，那么前缀 $[1, 13]$ 可以拆分成三个长度分别为 $8, 4, 1$ 的关键区间： $[1, 8], [9, 12], [13, 13]$ 。

按照这个规则，来看看从 $[1, 1]$ 到 $[1, 8]$ 是如何拆分的：

$[1, 1] = [1, 1]$	$(1 = 1)$
$[1, 2] = [1, 2]$	$(2 = 2)$
$[1, 3] = [1, 2] + [3, 3]$	$(3 = 2 + 1)$
$[1, 4] = [1, 4]$	$(4 = 4)$
$[1, 5] = [1, 4] + [5, 5]$	$(5 = 4 + 1)$
$[1, 6] = [1, 4] + [5, 6]$	$(6 = 4 + 2)$
$[1, 7] = [1, 4] + [5, 6] + [7, 7]$	$(7 = 4 + 2 + 1)$
$[1, 8] = [1, 8]$	$(8 = 8)$

数一数，按照这种拆分方式，总共有多少个不同的关键区间？（提示：把注意力放在区间的右端点上。）

有 8 个： $[1, 1], [1, 2], [3, 3], [1, 4], [5, 5], [5, 6], [7, 7], [1, 8]$ 。

一般地：

- 如果 i 是 2 的幂，那么 $[1, i]$ 无需拆分。
- 如果 i 不是 2 的幂，那么先拆分出一个最小的 2 的幂，记作 $lowbit(i)$ （例如 6 拆分出 2 ），得到长为 $lowbit(i)$ 的关键区间 $[i - lowbit(i) + 1, i]$ ，问题转换成剩下的 $[1, i - lowbit(i)]$ 如何拆分，这是一个规模更小的子问题。

由于关键区间的右端点互不相同，我们可以把右端点为 i 的关键区间的元素和保存在 $tree[i]$ 中

想要求解 $[1, i]$ 的区间和，先求出 $[i - lowbit(i), i]$ ，然后更新 i 为 $i - lowbit(i)$ 表示去找 $[1, i - lowbit(i)]$ 的区间和，使用一个 s 变量在外部累加。

而对于更新操作除了把对应的点更新外，还要去更新会受到影响区间，其中第一个区间就是 $[1, i]$ 而下一个要更新的区间为 $[i + 1, i + lowbit(i)]$ 就这样一直找下去

$query(i)$ 的结果是小于等于 $i - 1$ 的有多少个，数组要开 $n + 2$ 的因为想要取到数字 n 并且前缀和要做一位偏移

简略版板子如下，只能实现区间求和且数组数值数组，使用时像前缀和的操作方法

```
1 class Tree:
2
3     def __init__(self, nums: List[int]):
4         # 初始化，相当把每个点更新一次
5         n = len(nums)
6         self.nums = [0] * n
7         self.tree = [0] * (n + 1)
8         for i, x in enumerate(nums):
9             self.update(i, x)
10
```

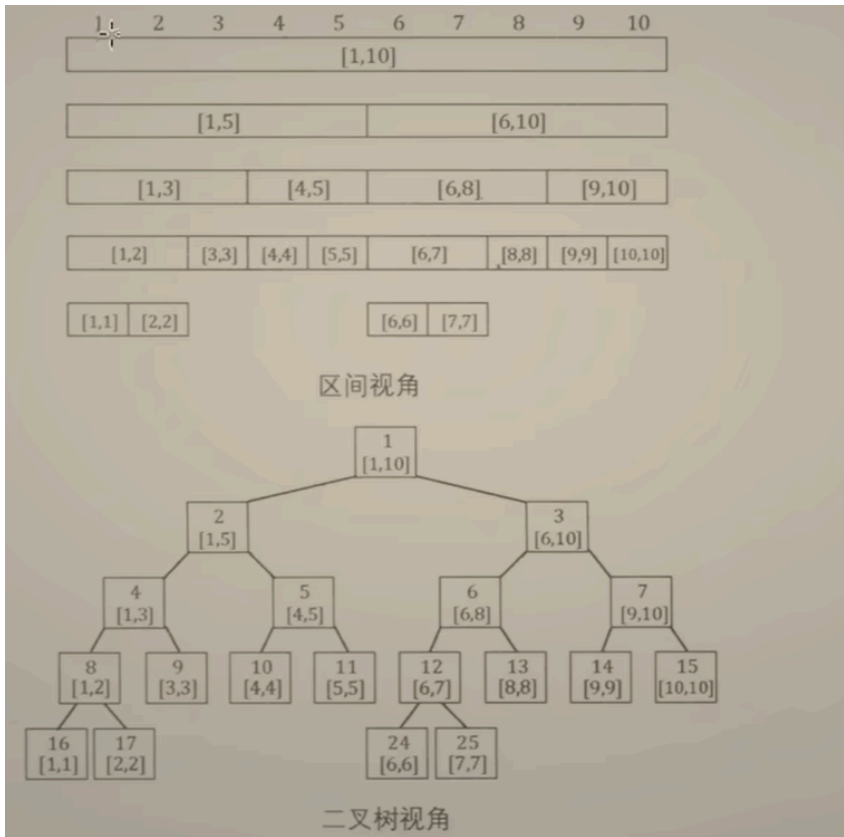
```
11 # 修改索引i位置为val，索引从0开始
12 def update(self, index: int, val: int) -> None:
13     # 找出差值用于更新前缀
14     diff = val - self.nums[index]
15     # 更新对应的位置
16     self.nums[index] = val
17     # 找区间，注意区间会多一位偏移
18     i = index + 1
19     # 迭代
20     while i < len(self.tree):
21         self.tree[i] += diff
22         # 结论，更新
23         i += i & -i
24     # 求前i个数字的和
25 def preSum(self, i):
26     # 累加
27     s = 0
28     while i:
29         s += self.tree[i]
30         i -= i & -i
31     return s
32 # 求区间[l,r]的和，索引从0开始
33 def sumRange(self, left: int, right: int) -> int:
34     # 左右位置，求解方式类似于一维
35     return self.preSum(right + 1) - self.preSum(left)
```

##

线段树

线段树解决区间和的问题，并且区间会被修改。线段数的两个操作：求区间和，修改区间，时间复杂度均为 $\log n$

模板



将区间划分至多 $4n$ (因为满二叉树的节点个数为 $2n-1$ ，一个基于经验的安全值是 $4n$) 个节点，其中每个节点记录的是一个区间的结果(和，最大值等等)

查询过程通过分支划分较小的区间，但不一定会划分到叶子节点，如果当前节点对应的区间范围已经被查询的范围包括了，就直接返回该节点储存的结果，而不需要再往下划分了，这样可以节省时间。

更新的过程中通过分治不断划分出较小的子区间，但如果要划分到叶子节点再更新，则在范围较大时会涉及到大量节点耗时较多，因此使用懒标记(延迟更新)，即更新操作推迟到真正需要的时候执行，每个节点会配备一个懒标记记录该节点上未处理的更新操作，当对该节点对应的区间进一步查询或更新时，会将标记下推，这样避免了对每个叶子节点的操作。

最基本结构

```
1 # 单点修改
2 def add(o, l, r, idx, val):
3     # 到了叶子节点
4     if l==r:
5         sum[o]+=val
6         return
7     mid=(l+r)>>1
```

```
8 # 在左子树
9 if idx<=mid:
10     add(2*o, l, mid, idx, val)
11 else: # 右子树
12     add(2*o+1, mid+1, r, idx, val)
13 # 子树递归完了需要更新更大的区间
14 sum[o]+=sum[2*o]+sum[2*o+1]
15
16 # L,R表示实际求解的区间
17 def query(o, l, r, L, R):
18     # 搜索区间被覆盖，直接返回结果，无需要划分下去
19     if L<=l and r<=R:
20         return sum[o]
21     mid=(l+r)>>1
22     res=0
23     # 左子树存在要计算的区间
24     if L<=mid:
25         res+=query(2*o, l, mid, L, R)
26     if R>=mid+1: # 右子树存在
27         res+=query(2*o+1, mid+1, r, L, R)
28     return res
```

涉及到区间更新据需要用到懒标记了

线段树数组实现 (数组版本，懒标记，区间查询/修改)，根据题目要求传入不同的函数：

```
1 class SegTree:
2     def __init__(self, nums, func):
3         self.nums = nums
4         self.n = len(nums)
5         self.tree = [0] * (4 * self.n) # 存储节点，4n是基于经验的安全值
6         self.lazy = [0] * (4 * self.n) # 懒标记数组和节点数组一样大
7         self.func = func # 操作的函数，求最大值，求和之类的
8         if self.n > 0:
9             self._build(0, 0, self.n - 1)
10        # 构建节点数组
11        def _build(self, root, l, r):
12            if l == r:
13                self.tree[root] = self.nums[r]
14                return
15            mid = (l + r) >> 1
16            # 递归更新左右，然后合并
17            self._build(root * 2 + 1, l, mid)
18            self._build(root * 2 + 2, mid + 1, r)
19            self.tree[root] = self.func(self.tree[root * 2 + 1], self.tree[root
20            * 2 + 2])
21
22        def _push_down(self, root, l, r):
23            """将懒标记下推到子节点"""
24            if self.lazy[root] != 0:
25                mid = (l + r) >> 1
26                # 更新左右子节点的值
```

```
26         self.tree[root * 2 + 1] += self.lazy[root] * (mid - l + 1)
27         self.tree[root * 2 + 2] += self.lazy[root] * (r - mid)
28         # 传递懒标记到子节点
29         self.lazy[root * 2 + 1] += self.lazy[root]
30         self.lazy[root * 2 + 2] += self.lazy[root]
31         # 清除当前节点的懒标记
32         self.lazy[root] = 0
33
34     def update(self, ul, ur, diff):
35         """区间更新，将区间 [ul, ur] 的值增加 diff"""
36         self._update_range(0, 0, self.n - 1, ul, ur, diff)
37
38         # 更新区间，root表示节点编号，可以从0或者1开始(这里从0开始，如果从1开始则左节点20,右
节点20+1)，lr是该节点覆盖的范围，LR是实际操作的范围
39     def _update_range(self, root, l, r, ul, ur, diff):
40         if ul <= l and r <= ur: # 当前区间完全在目标区间内，无需下推直接在这一层标记
41             self.tree[root] += diff * (r - l + 1) # 更新当前区间的值
42             self.lazy[root] += diff # 标记懒更新
43             return
44         self._push_down(root, l, r) # 下推懒标记
45         mid = (l + r) >> 1
46         # 涉及到当前节点的左儿子
47         if ul <= mid:
48             self._update_range(root * 2 + 1, l, mid, ul, ur, diff)
49         # 右儿子
50         if ur > mid:
51             self._update_range(root * 2 + 2, mid + 1, r, ul, ur, diff)
52         # 更新当前节点的值
53         self.tree[root] = self.func(self.tree[root * 2 + 1], self.tree[root
* 2 + 2])
54
55     def query(self, ql, qr):
56         """区间查询，返回 [ql, qr] 的值"""
57         return self._query_range(0, 0, self.n - 1, ql, qr)
58
59     # 参数同理
60     def _query_range(self, root, l, r, ql, qr):
61         if ql <= l and r <= qr: # 当前区间完全在目标区间内，无需继续划分直接返回
62             return self.tree[root]
63         self._push_down(root, l, r) # 下推懒标记
64         mid = (l + r) >> 1
65         res = 0
66         # 划分区间
67         if ql <= mid:
68             res = self.func(res, self._query_range(root * 2 + 1, l, mid, ql,
qr))
69         if qr > mid:
70             res = self.func(res, self._query_range(root * 2 + 2, mid + 1, r,
ql, qr))
71         return res
72
```

线段树模板（动态开点，区间和）需要注意当区间较大时(10**9)才会用到动态开点线段树，这个版本的实现很慢：

```
1 class Node:
2     def __init__(self):
3         self.left=None
4         self.right=None
5         self.add=0
6         self.val=0
7     def pushup(node):
8         node.val=node.left.val+node.right.val
9
10    def pushdown(node,lc,rc):
11        if not node.left:node.left=Node()
12        if not node.right:node.right=Node()
13
14        if node.add==0:return
15        node.left.val+=lc*node.add
16        node.right.val+=rc*node.add
17        # 懒标记不需要加
18        node.left.add += node.add
19        node.right.add += node.add
20        node.add=0
21
22    def update(root,start,end,l,r,val):
23        if l<=start and r>=end:
24            root.val+=(end-start+1)*val
25            root.add+=val
26            return
27        mid=(start+end)>>1
28        pushdown(root,mid-start+1,end-mid)
29        if l<=mid:update(root.left,start,mid,l,r,val)
30        if r>=mid+1:update(root.right,mid+1,end,l,r,val)
31        pushup(root)
32    def query(root,start,end,l,r):
33        if l <= start and r >= end:return root.val
34        mid=(start+end)>>1
35        ans=0
36        pushdown(root,mid-start+1,end-mid)
37        if l<=mid:ans+=query(root.left,start,mid,l,r)
38        if r>=mid+1:ans+=query(root.right,mid+1,end,l,r)
39        return ans
```

注意根据题意的不同线段树中的各个函数也是可能改变的，基本上pushup,pushdown 之类的函数会变化

##

字典树

Trie字符串统计

维护一个字符串集合，支持两种操作：

- 1. **I x** 向集合中插入一个字符串 x ；
- 2. **Q x** 询问一个字符串在集合中出现了多少次。

共有 N 个操作，所有输入的字符串总长度不超过 10^5 ，字符串仅包含小写英文字母。

输入格式

第一行包含整数 N ，表示操作数。

接下来 N 行，每行包含一个操作指令，指令为 **I x** 或 **Q x** 中的一种。

输出格式

对于每个询问指令 **Q x**，都要输出一个整数作为结果，表示 x 在集合中出现的次数。

每个结果占一行。

数据范围

$1 \leq N \leq 2 * 10^4$

输入样例：

```
5
I abc
Q abc
Q ab
I ab
Q ab
```

输出样例：

```
1
0
1
```

```
1 class Trie:
2     def __init__(self):
3         self.children=[None]*26
4         self.is_end=0
5     def insert(self,s):
6         d=self
7         for c in s:
8             c=ord(c)-ord('a')
9             if not d.children[c]:
10                 d.children[c]=Trie()
11             d=d.children[c]
12         d.is_end+=1
13     def search(self,s):
14         d=self
15         for c in s:
16             c=ord(c)-ord('a')
17             if not d.children[c]:
18                 return 0
19             d=d.children[c]
20         return d.is_end
21 q=int(input())
22 tree=Trie()
23 while q:
24     a,s=input().split()
```

```
25     if a=='I':
26         tree.insert(s)
27     else:
28         print(tree.search(s))
29     q-=1
```

二维前缀和

二维区域和检索-矩阵不可变

给定一个二维矩阵 `matrix`，以下类型的多个请求：

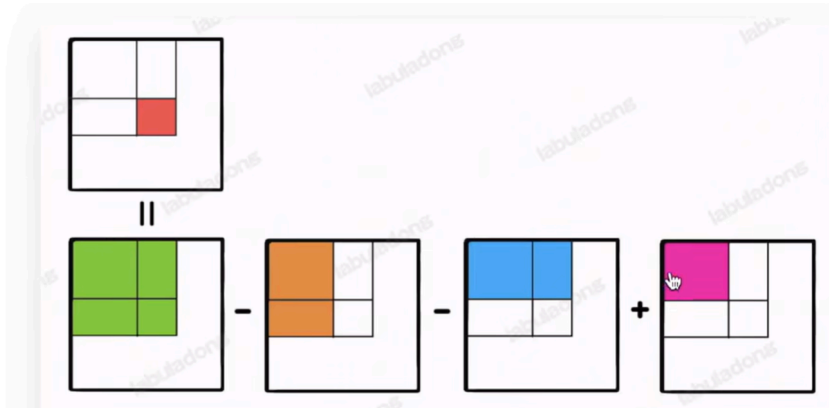
- 计算其子矩形范围内元素的总和，该子矩阵的 **左上角** 为 `(row1, col1)`，**右下角** 为 `(row2, col2)`。

实现 `NumMatrix` 类：

- `NumMatrix(int[][] matrix)` 给定整数矩阵 `matrix` 进行初始化
- `int sumRegion(int row1, int col1, int row2, int col2)` 返回 **左上角** `(row1, col1)`、**右下角** `(row2, col2)` 所描述的子矩阵的元素 **总和**。

利用前缀和数组，计算出从原点到*i,j*的矩阵元素和

- 1.从 `(row2, col2)` 处的前缀和减去 `(row1 - 1, col2)` 处的前缀和（如果 `row1` 不为0的话）。
- 2.再减去 `(row2, col1 - 1)` 处的前缀和（如果 `col1` 不为0的话）。
- 3.最后再加上 `(row1 - 1, col1 - 1)` 处的前缀和（如果 `row1` 和 `col1` 都不为0的话）。



```
1 class NumMatrix(object):
2
3     def __init__(self, matrix):
4         if not matrix or not matrix[0]:
5             self.prefix_sum = None
6             return
7         self.matrix=matrix
8         m=len(matrix)
9         n=len(matrix[0])
```

```
10 # 所构建的数组长宽都要多出一，可以方便计算，不是防止越界
11 self.prematrix=[0]*(n+1) for _ in range(m+1)]
12 for i in range(1,m+1):
13     for j in range(1,n+1):
14         # 子矩阵的元素和为矩阵元素加上左边的子矩阵加上上边的子矩阵减去重复的左上
        矩阵
15         self.prematrix[i][j]=self.matrix[i-1][j-1]+self.prematrix[i-1][j]+self.prematrix[i][j-1]-self.prematrix[i-1][j-1]
16
17 def sumRegion(self, row1, col1, row2, col2):
18     # 最后的计算中注意不能重叠，放到矩阵上做一位偏移
19     return self.prematrix[row2+1][col2+1]-self.prematrix[row2+1][col1]-self.prematrix[row1][col2+1]+self.prematrix[row1][col1]
```

二维差分

输入一个 n 行 m 列的整数矩阵，再输入 q 个操作，每个操作包含五个整数 x_1,y_1,x_2,y_2,c ，其中 (x_1,y_1) 和 (x_2,y_2) 表示一个子矩阵的左上角坐标和右下角坐标。

每个操作都要将选中的子矩阵中的每个元素的值加上 c 。

请你将进行完所有操作后的矩阵输出。

输入格式

第一行包含整数 n,m,q 。

接下来 n 行，每行包含 m 个整数，表示整数矩阵。

接下来 q 行，每行包含 5 个整数 x_1,y_1,x_2,y_2,c ，表示一个操作。

输出格式

共 n 行，每行 m 个整数，表示所有操作进行完毕后的最终矩阵。

数据范围

- $1 \leq n,m \leq 1000$,
- $1 \leq q \leq 100000$,
- $1 \leq x_1 \leq x_2 \leq n$,
- $1 \leq y_1 \leq y_2 \leq m$,
- $-1000 \leq c \leq 1000$,
- $-1000 \leq$ 矩阵内元素的值 ≤ 1000

二维差分 要构造比给定矩阵大2的差分矩阵，初始化为0，因为方便构建前缀和操作需要n+1（在原矩阵上构建前缀和），而差分的更新操作又需要n+1。坐标都要做一位偏移，而右侧下侧的坐标额外做一次偏移

然后根据原矩阵构造差分，对于原矩阵的每一个值，相当于在差分矩阵中插入一个值，不过左上角和右下角是一样的。差分矩阵的更新操作为对(i,j,x,y)把i,j位置加一，最后的前缀和使得以i,j为左上角到最右下的矩阵面积中的值都变化，因此需要在i,y+1和x+1,j减去对应的值，而在x+1,y+1处加上多减去的部分。

```
1 from itertools import accumulate as ac
2 n,m,q=[int(x) for x in input().split()]
```

```
3 li=[]
4 for _ in range(n):
5     li.append([int(x) for x in input().split()])
6 # 矩阵规模大出2
7 diff=[0 for _ in range(m+2)] for _ in range(n+2)]
8
9 # 根据原矩阵初始化差分矩阵
10 for i in range(n):
11     for j in range(m):
12         c=li[i][j]
13         # 与一般的更新操作一样
14         diff[i+1][j+1]+=c
15         diff[i+1][j+2]-=c
16         diff[i+2][j+1]-=c
17         diff[i+2][j+2]+=c
18
19
20 while q:
21     # 更新
22     x1,y1,x2,y2,c=[int(x)-1 for x in input().split()]
23     c+=1
24     diff[x1+1][y1+1]+=c
25     diff[x1+1][y2+2]-=c
26     diff[x2+2][y1+1]-=c
27     diff[x2+2][y2+2]+=c
28     q-=1
29
30 # 构建二维前缀和
31 for i in range(1,n+1):
32     for j in range(1,m+1):
33         diff[i][j]+=diff[i-1][j]+diff[i][j-1]-diff[i-1][j-1]
34
35 # 减去无用的第一行最后一行第一列最后一列
36 for i in range(n+2):
37     diff[i]=diff[i][1:-1]
38 diff=diff[1:-1]
39
40 for d in diff:
41     print(*d,end=' ')
42     print()
```

并查集

该算法主要用于解决图论中的动态连通性问题。

```
1 # 完整模板
2 # 迭代方式的find函数要配合size使用
3 class UF:
4     def __init__(self, n):
5         self.count = n
6         self.parent = [i for i in range(n)]
7         self.size = [1 for _ in range(n)]
```

```
8
9
10 def union(self, p, q):
11     rootp = self.find(p)
12     rootq = self.find(q)
13     if rootq == rootp:
14         return
15     # 小的连接到大的上面去
16     if self.size[rootq] < self.size[rootp]:
17         self.parent[rootq] = rootp
18         self.size[rootp] += self.size[rootq]
19     else:
20         self.parent[rootp] = rootq
21         self.size[rootq] += self.size[rootp]
22     self.count -= 1
23
24 def connect(self, p, q):
25     rootp = self.find(p)
26     rootq = self.find(q)
27     return rootq == rootp
28
29 # 会将节点铺开理解到一个节点上，保证数树的深度为一，给一个连通块理解上一个新的连通块并不改变原有的根节点
30
31 def find(self,p):
32     if self.parent[p]!=p:
33         self.parent[p]=self.find(self.parent[p])
34     return self.parent[p]
```

Logtrick

利用运算结果是有限的(位运算，乘积等于和，公因数)，来维护一个较短的数组(如果求得的是一个最接近的值那么可以直接往回找，如果求的是子数组的长度，那么额外维护一个索引值，如果是个数，那么就维护一个个数，在去重的时候更新或合并)，这个数组是当前位置之前的所有子数组的运算结果，每次枚举一个值的时候都会遍历这个数组重新计算以当前值为端点得到的新的子数组的结果。

按位或最大的最小子数组长度

给你一个长度为 n 下标从 0 开始的数组 `nums`，数组中所有数字均为非负整数。对于 0 到 $n - 1$ 之间的每一个下标 i ，你需要找出 `nums` 中一个 **最小** 非空子数组，它的起始位置为 i （包含这个位置），同时有 **最大** 的 **按位或运算** 值。

- 换言之，令 B_{ij} 表示子数组 `nums[i...j]` 的按位或运算的结果，你需要找到一个起始位置为 i 的最小子数组，这个子数组的按位或运算的结果等于 $\max(B_{ik})$ ，其中 $i \leq k \leq n - 1$ 。

一个数组的按位或运算值是这个数组里所有数字按位或运算的结果。

请你返回一个大小为 n 的整数数组 `answer`，其中 `answer[i]` 是开始位置为 i ，按位或运算结果最大，且 **最短** 子数组的长度。

子数组 是数组里一段连续非空元素组成的序列。

示例 1:

输入: `nums = [1,0,2,1,3]`
输出: `[3,3,2,2,1]`
解释:
任何位置开始，最大按位或运算的结果都是 3 。
- 下标 0 处，能得到结果 3 的最短子数组是 `[1,0,2]`。
- 下标 1 处，能得到结果 3 的最短子数组是 `[0,2,1]`。
- 下标 2 处，能得到结果 3 的最短子数组是 `[2,1]`。
- 下标 3 处，能得到结果 3 的最短子数组是 `[1,3]`。
- 下标 4 处，能得到结果 3 的最短子数组是 `[3]`。
所以我们返回 `[3,3,2,2,1]`。

二刷。

示例 2:

输入: `nums = [1,2]`
输出: `[2,1]`
解释:
下标 0 处，能得到最大按位或运算值的最短子数组长度为 2 。
下标 1 处，能得到最大按位或运算值的最短子数组长度为 1 。
所以我们返回 `[2,1]`。

提示:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^9$

题意为：找出以每个元素为起始位置的子数组中，按位或能得到最大值的最短子数组。

异或运算得到的子数组只会是递增或是不变的具有单调性，使用一个数组来维护每个位置的元素所能得到的最大值以及这个位置，百万级别的数据对应到二进制上最多只有三十几个状态，因此维护的数据不会太大，而每个元素能对应的区间又可以由前一个元素推出。

```
1 class Solution:
2     def smallestSubarrays(self, nums: List[int]) -> List[int]:
3         n=len(nums)
4         ans=[0]*n
5         # 维护状态（异或值，右端点索引），每次加到尾部，因此数组首部就是最大的异或值
```

```
6  ors=[]
7  # 倒序遍历，因为要以i为起点
8  for i in range(n-1,-1,-1):
9      ors.append([0,i])
10     # 每个元素所有子数组的结果可以由上一个元素的结果推出，因为运算具有单调性
11     for p in ors:
12         p[0]|=nums[i]
13     # 以下是去重的逻辑，因为对于相同的值，题目要求去最短区间
14     l=0
15     r=1
16     while r<len(ors):
17         if ors[r][0]!=ors[l][0]:
18             l+=1
19             ors[l]=ors[r]
20         else:
21             # 对于相同的值取小的右端点
22             ors[l][1]=ors[r][1]
23             r+=1
24     del ors[l+1:]
25     # 可以保证数组中的第一个元素是当前能到达的最大的
26     ans[i]=ors[0][1]-i+1
27 return ans
28
```

子数组按位或操作

我们有一个非负整数数组 `arr` 。

对于每个（连续的）子数组 `sub = [arr[i], arr[i + 1], ..., arr[j]]` （`i <= j`），我们对 `sub` 中的每个元素进行按位或操作，获得结果 `arr[i] | arr[i + 1] | ... | arr[j]` 。

返回可能结果的数量。多次出现的结果在最终答案中仅计算一次。

示例 1:

输入: `arr = [0]`
输出: 1
解释:
只有一个可能的结果 `0` 。

示例 2:

输入: `arr = [1,1,2]`
输出: 3
解释:
可能的子数组为 `[1]`, `[1]`, `[2]`, `[1, 1]`, `[1, 2]`, `[1, 1, 2]`。
产生的结果为 `1, 1, 2, 1, 3, 3`。
有三个唯一值，所以答案是 `3` 。

示例 3:

输入: `arr = [1,2,4]`
输出: 6
解释:
可能的结果是 `1, 2, 3, 4, 6`, 以及 `7` 。

同一道题一样的解法，在找一个子数组最大的或结果中记录结果。或实质上是并集，遍历每一个元素，求他和之前的元素之间的或的结果，如果if `nums[i] | nums[j]==nums[j]`就说明当前的元素不会使前面的集合变大了直接退出

```
1  class Solution:
2      def subarrayBitwiseORs(self, nums: List[int]) -> int:
3          ans=0
4          n=len(nums)
5          memo=set()
6          # 这个题目不需要记录索引直接用集合去重
7          ors=set()
8          for i in range(n-1,-1,-1):
9              ors.add(nums[i])
10             temp=set()
11             # 不能直接对原集合操作
12             for j in ors:
13                 j|=nums[i]
14                 if j not in memo:
15                     ans+=1
16                     memo.add(j)
17                     temp.add(j)
18             ors=temp
19         return ans
```


倍增

Teleporter

题意翻译

高桥王国有 N 个城镇，编号从 $1 \sim N$ 。

每一个城镇有一个传送器。在 i 号城镇的传送阵可以是你到达城镇 A_i 。

国王高桥喜欢正整数 K 。自私的国王想要知道他 从第 1 号城镇出发，用 K 次传送阵可以到达的位置。

请帮国王写一个程序完成这个问题。

by [djh123456](#)。

输入输出样例

输入 #1	复制	输出 #1	复制
4 5 3 2 4 1		4	
输入 #2	复制	输出 #2	复制
6 727202214173249351 6 5 2 5 3 2		2	

说明/提示

制約

- $2 \leq N \leq 2 \times 10^5$
- $1 \leq A_i \leq N$
- $1 \leq K \leq 10^{18}$

给定的K非常大，唯一的做法是nlogk，定义fi,j表示从第i个城市经过2^j后所在的城市，这个结果可以由f[i][j-1][j-1]求出，即i位置经过了2^(j-1)步后再走2^(j-1)步求出。这样选定的状态j最多只有65。而题目所求的第一个位置在k步后所在的位置可以拆位递推求解。

另一种写法是找周期取模

```
1 n,k=RR()
2 nums=RR()
3 K=66
4
5 f=[[0]*K for _ in range(n)]
6 # 初始化
7 for i in range(n):
8     f[i][0]=nums[i]-1
9
10 # 递推过程
11 for j in range(1,K):
```

```
12     for i in range(n):
13         f[i][j]=f[f[i][j-1]][j-1]
14
15 ans=j=0
16 # 对于k=11也就是1011拆分为8+2+1，模拟这个递推过程
17 while k:
18     if k&1:ans=f[ans][j]
19     j+=1
20     k>>=1
21
22 print(ans+1)
23
```

##

试填法

位运算经典算法，本质上是对每一位去构造答案

给定操作次数内使剩余元素的或值最小

给你一个下标从 **0** 开始的整数数组 `nums` 和一个整数 `k` 。

一次操作中，你可以选择 `nums` 中满足 $0 \leq i < \text{nums.length} - 1$ 的一个下标 `i`，并将 `nums[i]` 和 `nums[i + 1]` 替换为数字 `nums[i] & nums[i + 1]`，其中 `&` 表示按位 AND 操作。

请你返回 **至多** `k` 次操作以内，使 `nums` 中所有剩余元素按位 OR 结果的 **最小值**。

示例 1:

输入: `nums = [3,5,3,2,7]`, `k = 2`
输出: 3
解释: 执行以下操作:
1. 将 `nums[0]` 和 `nums[1]` 替换为 `(nums[0] & nums[1])`，得到 `nums` 为 `[1,3,2,7]`。
2. 将 `nums[2]` 和 `nums[3]` 替换为 `(nums[2] & nums[3])`，得到 `nums` 为 `[1,3,2]`。
最终数组的按位或值为 3。
3 是 `k` 次操作以内，可以得到的剩余元素的最小按位或值。

示例 2:

输入: `nums = [7,3,15,14,2,8]`, `k = 4`
输出: 2
解释: 执行以下操作:
1. 将 `nums[0]` 和 `nums[1]` 替换为 `(nums[0] & nums[1])`，得到 `nums` 为 `[3,15,14,2,8]`。
2. 将 `nums[0]` 和 `nums[1]` 替换为 `(nums[0] & nums[1])`，得到 `nums` 为 `[3,14,2,8]`。
3. 将 `nums[0]` 和 `nums[1]` 替换为 `(nums[0] & nums[1])`，得到 `nums` 为 `[2,2,8]`。
4. 将 `nums[1]` 和 `nums[2]` 替换为 `(nums[1] & nums[2])`，得到 `nums` 为 `[2,0]`。
最终数组的按位或值为 2。
2 是 `k` 次操作以内，可以得到的剩余元素的最小按位或值。

提示:

- `1 <= nums.length <= 105`
- `0 <= nums[i] < 230`
- `0 <= k < nums.length`

要求最后或出的结果最小，那么所有数字最高位的1都应该尝试消除掉，通过试填法来构造答案，从高位到低位枚举

按照题目要求两两相与，如果一段数字能与出0，那么操作次数就是这段数字长度-1，否则就需要从外部引入一个0，操作次数就是这段数字的长度。如果把这一位变为0需要的次数大于k，就说明无论如何都无法将这一位变为0，遂不考虑这位，否则在找更低位的时候仍需保持这一位为0的约束。

```
1 class Solution:
2     def minOrAfterOperations(self, nums: List[int], k: int) -> int:
3         ans=0
4         # mask用于取出每个数字的约束位，比如要考虑第一位和第三位，那么mask就为101，将每个数的一三位取出，第二位不考虑因为无法变为0，通过mask在找低位时可以同时考虑到高位的约束
5         mask=0
```

```
6         # 操作每一位
7         for i in range(29,-1,-1):
8             # ans_res是用于计算按位与的结果，初始时应该为11111...，这里令为-1可以简单
            表示
9             ans_res=-1
10            # 先要取出这一位
11            mask|=(1<<i)
12            # 记录操作次数
13            cnt=0
14            # 从头开始找每一段
15            for v in nums:
16                # 取出约束位
17                v&=mask
18                # 按位与
19                ans_res&=v
20                # 合并x操作次数加一，因为k小于数组长度，所以如果一直找不到的话最后cnt的
            次数一定会大于k
21            if ans_res:
22                cnt+=1
23            else:
24                # 这一段操作成功，那么去找下一段
25                ans_res=-1
26            # 无法变为1
27            if cnt>k:
28                # 加入到答案中
29                ans|=(1<<i)
30                # 撤销mask上这一位
31                mask^=(1<<i)
32            return ans
```

位运算常用技巧

集合与集合

术语	集合	位运算	举例	举例
交集	$A \cap B$	$a \& b$	$\{0, 2, 3\} \cap \{0, 1, 2\} = \{0, 2\}$	1101 & 0111 = 0101
并集	$A \cup B$	$a \mid b$	$\{0, 2, 3\} \cup \{0, 1, 2\} = \{0, 1, 2, 3\}$	1101 0111 = 1111
对称差	$A \Delta B$	$a \oplus b$	$\{0, 2, 3\} \Delta \{0, 1, 2\} = \{1, 3\}$	1101 ⊕ 0111 = 1010
差	$A \setminus B$	$a \& \sim b$	$\{0, 2, 3\} \setminus \{1, 2\} = \{0, 3\}$	1101 & 1001 = 1001
差 (子集)	$A \setminus B \quad (B \subseteq A)$	$a \oplus b$	$\{0, 2, 3\} \setminus \{0, 2\} = \{3\}$	1101 ⊕ 0101 = 1000
包含于	$A \subseteq B$	$a \& b = a$ $a \mid b = b$	$\{0, 2\} \subseteq \{0, 2, 3\}$	0101&1101 = 0101 0101 1101 = 1101

集合与元素

$i > n < n$ 可以取出n位之上的

二进制取反就是找补集的操作：异或上 $((1 \ll n) - 1)$ ，n表示二进制长度，如果不想把最高位取反那么把n减去1即可。

lowbit就是二进制下最低位1与其右边的0构成数字大小，可以通过 $i \& -i$ 求出

对于二进制加法有 $a + b = (a \wedge b) + 2(a \& b)$

术语	集合	位运算	举例	举例
空集	$\varnothing$	0		
单元素集合	$\{i\}$	$1 \ll i$	$\{2\}$	$1 \ll 2$
全集	$U = \{0, 1, 2, \dots, n - 1\}$	$(1 \ll n) - 1$	$\{0, 1, 2, 3\}$	$(1 \ll 4) - 1$
补集	$\complement_U S = U \setminus S$	$\sim s$ 或者 $((1 \ll n) - 1) \oplus s$		
属于	$i \in S$	$(s \gg i) \& 1 = 1$	$2 \in \{0, 2, 3\}$	$(1101 \gg 2) \& 1 = 1$
不属于	$i \notin S$	$(s \gg i) \& 1 = 0$	$1 \notin \{0, 2, 3\}$	$(1101 \gg 1) \& 1 = 0$
添加元素	$S \cup \{i\}$	$s \mid (1 \ll i)$	$\{0, 3\} \cup \{2\}$	$1001 \mid (1 \ll 2)$
删除元素	$S \setminus \{i\}$	$s \& \sim (1 \ll i)$	$\{0, 2, 3\} \setminus \{2\}$	$1101 \& \sim (1 \ll 2)$
删除元素 (一定在集合中)	$S \setminus \{i\} \quad (i \in S)$	$s \oplus (1 \ll i)$	$\{0, 2, 3\} \setminus \{2\}$	$1101 \oplus (1 \ll 2)$
删除最小元素		$s \& (s - 1)$		见下

术语	Python	Java	C++	Go
集合大小 (元素个数)	<code>s.bit_count()</code>	<code>Integer.bitCount(s)</code>	<code>__builtin_popcount(s)</code>	<code>bits.OnesCount(s)</code>
二进制长度 (减一得到集合中的最大元素)	<code>s.bit_length()</code>	<code>Integer.numberOfLeadingZeros(s)</code>	<code>32 - __builtin_clz(s)</code>	<code>bits.Len(s)</code>
集合中的最小元素	<code>(s & -s).bit_length() - 1</code>	<code>Integer.numberOfTrailingZeros(s)</code>	<code>__builtin_ctz(s)</code>	<code>bits.TrailingZeros(s)</code>

遍历集合

直接循环

```
1 for i in range(n):
2     if (s >> i) & 1: # i 在 s 中
3         # 处理 i 的逻辑
```

使用lowbit的技巧

```
1 t = s
2 while t:
3     lowbit = t & -t
4     t ^= lowbit
5     i = lowbit.bit_length() - 1
6     # 处理 i 的逻辑
```

枚举集合

最直接的是for循环

所有子集

```
1 sub = s
2 while True:
3     # 处理 sub 的逻辑
4     sub = (sub - 1) & s
5     if sub == s:
6         break
```

从大到小枚举所有子集，不包括空集

```
1 sub = s
2 while sub:
3     # 处理 sub 的逻辑
4     sub = (sub - 1) & s
```

树的直径

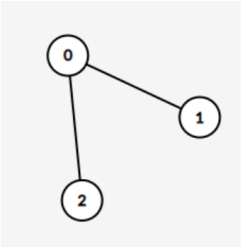
树的直径

给你这棵「无向树」，请你测算并返回它的「直径」：这棵树上最长简单路径的 **边数**。

我们用一個由所有「边」组成的数组 `edges` 来表示一棵无向树，其中 `edges[i] = [u, v]` 表示节点 `u` 和 `v` 之间的双向边。

树上的节点都已经用 `{0, 1, ..., edges.length}` 中的数做了标记，每个节点上的标记都是独一无二的。

示例 1:



输入: `edges = [[0,1],[0,2]]`
输出: 2
解释: 这棵树上最长的路径是 1 - 0 - 2, 边数为 2。

示例 2:

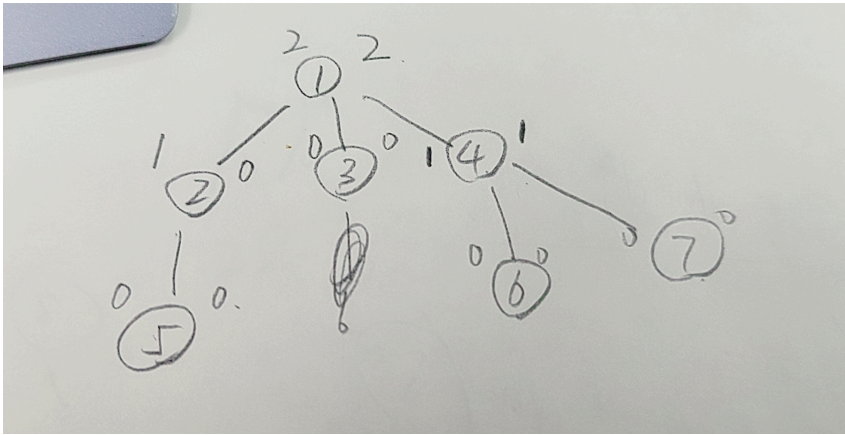
树的直径就是两个相邻最远的节点的距离

- BFS方法，从任意节点开始都可以得到任意一个极端节点（距离所有节点最远的两个节点），然后从这个节点开始再进行一次BFS就可以得到另一个极端节点，这两个极端节点之间的距离就是树的直径。

```
1 from collections import deque
2
3
4 class Solution(object):
5     def treeDiameter(self, edges):
6         if not len(edges):
7             return 0
8         # 使用set集合记录每一个边
9         paths = [set() for _ in range(len(edges) + 1)]
10        for edge in edges:
11            u, v = edge
12            paths[u].add(v)
13            paths[v].add(u)
14
15        # bfs框架
16        def bfs(start):
17            d = deque([start])
18
19            # 备忘录防止重复加入
20            visited = [False] * (len(paths))
21            visited[start] = True
22            # 记录最后一个节点
23            lastnode = None
```

```
24 # 距离从-1开始，因为遍历到最后一层时还会向外遍历一次，如果从0开始最后的结果
    会多出一
25     distance=-1
26     while d:
27         sz = len(d)
28         # 只遍历之前队列中元素的个数次
29         for _ in range(sz):
30             # 弹出
31             curr = d.popleft()
32             for i in paths[curr]:
33                 # 加入不重复的邻居
34                 if not visited[i]:
35                     visited[i] = True
36                     d.append(i)
37                     lastnode = i
38             # 距离加一
39             distance += 1
40         return [distance, lastnode]
41
42 distance, lastnode = bfs(0)
43 return bfs(lastnode)[0]
```

- 使用回溯算法解题，如果可以得到从父节点到任何后代叶子节点的最长距离t1,t2，那么经过该父节点的最长半径就是t1+t2，整个树的最长半径就是所有节点中的最长半径



```
1 class Solution(object):
2     def treeDiameter(self, edges):
3
4         paths = [set() for _ in range(len(edges) + 1)]
5         # 记录所有边
6         for edge in edges:
7             u, v = edge
8             paths[u].add(v)
9             paths[v].add(u)
10
11         # 设置备忘录
```

```
12 visited=[False]*len(paths)
13 # 不断更新外部变量来得到最大值
14 self.diameter=0
15
16 # 会返回以p为父节点到叶子节点的最长距离
17 def dfs(p,visited):
18     # 设置两个到叶子节点的最长距离
19     left = 0
20     right = 0
21     # 标记
22     visited[p]=True
23     for val in paths[p]:
24         # 包含过了
25         if visited[val]:
26             continue
27         # 该节点到叶子节点的距离为它的子节点到叶子节点的距离加一
28         distance=1+dfs(val,visited)
29         # 更新
30         if distance>left:
31             left,right=distance,left
32         elif distance>right:
33             right=distance
34
35     self.diameter=max(self.diameter,left+right)
36
37     return left
38
39 dfs(0,visited)
40 return self.diameter
```

矩形面积

矩形牛棚

作为一个资本家，农夫约翰希望通过购买更多的奶牛来扩大他的牛奶业务。

因此，他需要找地方建立一个新的牛棚。

约翰购买了一大块土地，这个土地可以看作是一个 R 行（编号 $1 \sim R$ ） C 列（编号 $1 \sim C$ ）的方格矩阵。

不幸的是，他发现其中的部分方格区域已经被破坏了，因此他无法在整个 $R \times C$ 的土地上建立牛棚。

经调查，他发现共有 P 个方格内的土地遭到了破坏。

建立的牛棚必须是矩形的，并且内部不能包含被破坏的土地。

请你帮约翰计算，他能建造的最大的牛棚的面积是多少。

输入格式

第一行包含三个整数 R, C, P 。

接下来 P 行，每行包含两个整数 r, c ，表示第 r 行第 c 列的方格区域内土地是被破坏的。

输出格式

输出牛棚的最大可能面积。

数据范围

$1 \leq R, C \leq 3000$,
 $0 \leq P \leq 30000$,
 $1 \leq r \leq R$,
 $1 \leq c \leq C$

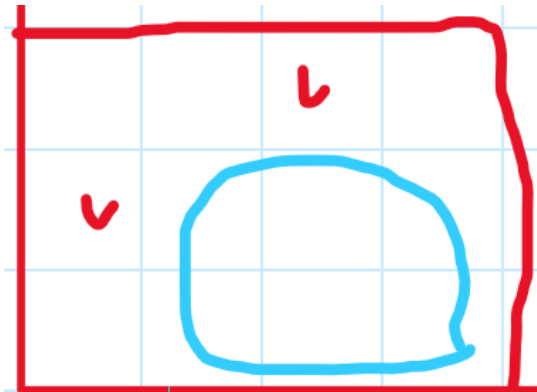
输入样例：

```
3 4 2
1 3
2 1
```

输出样例：

```
6
```

样例类似于：



枚举每一条下边界，求以这个下边界为底找最大的矩形面积，问题转换为直方图中的最大矩形，对于一条下边它每个位置上的直方高是多少使用递归求解如果这个位置上有一个破坏的点那么它的高是0，否则是上一行的高加上一。

```
1  from collections import deque
2  import copy
3  import sys
4  input=sys.stdin.readline
5
6  m, n, c = [int(x) for x in input().split()]
7
8  # 记录破坏的点
9  grid=[[0]*n for _ in range(m)]
10 # 记录每个位置上直方的高
11 h=[[0]*n for _ in range(m)]
12
13
14 for _ in range(c):
15     a, b = [int(x)-1 for x in input().split()]
16     grid[a][b]=1
17
18 # 迭代求解
19 h[0]=[v^1 for v in grid[0]]
20 for i in range(1,m):
21     for j in range(n):
22         if not grid[i][j]:h[i][j]=h[i-1][j]+1
23
24 ans = 0
25 # 枚举每个高
26 for b in range(m):
27     nums =h[b]
28     l = [0]*n
29     r = [0]*n
30     d = deque()
31     # 单调栈计算左右位置
32     for i in range(len(nums)):
33         v = nums[i]
34
```

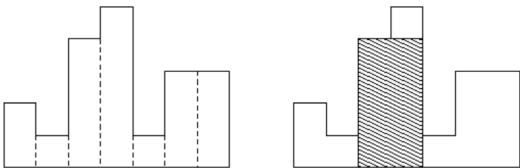
```
35     while d and nums[d[-1]] >= v:
36         d.pop()
37     if d:
38         l[i]=d[-1]
39     else:
40         l[i]=-1
41     d.append(i)
42 d = deque()
43 for i in range(n- 1, -1, -1):
44     v = nums[i]
45     while d and nums[d[-1]] >= v:
46         d.pop()
47     if d:
48         r[i]=d[-1]
49     else:
50         r[i]=n
51     d.append(i)
52
53 for i, v in enumerate(nums):
54     ans = max(ans, (v) * (r[i] - l[i]-1))
55 print(ans)
```

直方图中最大的矩形

直方图是由在公共基线处对齐的一系列矩形组成的多边形。

矩形具有相等的宽度，但可以具有不同的高度。

例如，图例左侧显示了由高度为 2, 1, 4, 5, 1, 3, 3 的矩形组成的直方图，矩形的宽度都为 1：



通常，直方图用于表示离散分布，例如，文本中字符的频率。

现在，请你计算在公共基线处对齐的直方图中最大矩形的面积。

图例右图显示了所描绘直方图的最大对齐矩形。

输入格式

输入包含几个测试用例。

每个测试用例占据一行，用以描述一个直方图，并以整数 n 开始，表示组成直方图的矩形数目。

然后跟随 n 个整数 $h_1, \dots, h_n$ 。

这些数字以从左到右的顺序表示直方图的各个矩形的高度。

每个矩形的宽度为 1。

同行数字用空格隔开。

当输入用例为 $n = 0$ 时，结束输入，且该用例不用考虑。

枚举每一个上边，以该上边形成的最大矩形取决于两边比它小的上边的位置，使用单调栈找出两边小于它的位置即可。

```
1 from math import inf
2 import sys
3 from collections import deque
4
5 input = sys.stdin.readline
6 while 1:
7     nums = [int(x) for x in input().split()][1:]
8     if len(nums) == 0:
9         break
10    # 储存左边小于的，右边小于的
11    l = []
12    r = []
13    # 栈中储存的是下标
14    d = deque()
15    ans = -inf
16    for i, v in enumerate(nums):
17        if not d:
18            l.append(-1)
19            d.append(i)
20            continue
21        # 大于的值没必要留在栈中，因为只会取到小的值
22        while d and nums[d[-1]] >= v:
23            d.pop()
24        if not d:
25            l.append(-1)
26        else:
27            l.append(d[-1])
28        d.append(i)
29    # 注意清空栈
30    d=deque()
31    for i in range(len(nums) - 1, -1, -1):
32        v = nums[i]
33        if not d:
34            r.append(len(nums))
35            d.append(i)
36            continue
37        while d and nums[d[-1]] >= v:
38            d.pop()
39        if not d:
40            r.append(len(nums))
41        else:
42            r.append(d[-1])
43        d.append(i)
44    r = list(reversed(r))
45    for i, v in enumerate(nums):
46        ans = max(ans, v * (r[i] - l[i] - 1))
47    print(ans)
```

